FIX adapter program interface

Administrator's manual

Version 4.2



Contents

1. General information 4
   1.1 Purpose 4
   1.2 Operating principles 4
   1.3 Connection scheme 5
2. Interface installation and configuration 5
   2.1 Technical requirements 5
   2.2 Users to operate with service under Linux 5
   2.3 Installation 6
   2.4 Service migration from Windows to Astra Linux 12
   2.5 Removing service 14
   2.6 quikfix.ini file format 16
   2.7 quikapi.ini file format 55
   2.8 Configuration of digital signature 57
   2.9 Configuration of FIX messages 58
   2.10 SecInfo.xml file format 69
   2.11 ClientEmailMessages.xml file format 70
   2.12 Methods of instrument identification 76
   2.13 Methods of trade account identification 77
   2.14 Methods of client code identification 78
   2.15 SecTypes.xml file format 78
3. Launch parameters 79
   3.1 -drop_history 79
   3.2 -new 79
   3.3 -v 79
   3.4 -rollout 80
4. Password encryption 80

You may email your comments on this Manual to: quiksupport@arqatech.com

1. General information

1.1 Purpose

The FIX Program Interface or **FIX adapter** (hereinafter Interface) provides of connecting external systems for trading via QUIK by means of FIX protocol.

FIX (Financial Information eXchange) is a specialized protocol for exchanging financial data in the online mode.

> **For information about specifics of external systems, contact QUIK technical support: quiksupport@arqatech.com.**

Interface functions:

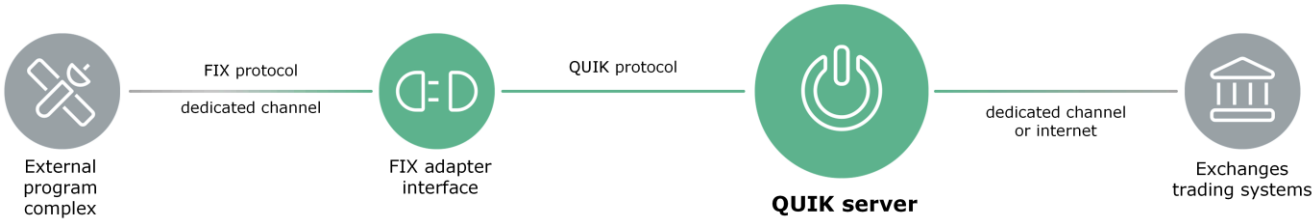
- The FIX adapter program interface is intended for connecting external applications to the QUIK server in order to obtain data and transmit trade instractions.
- The interface allows getting from the QUIK server market data, reference data, 'all trades' data as well as data on orders and trades forwarded and executed via this particular interface.
- The interface provides access to any trading system, trading mode or order type supported by a respective QUIK server as well as to the full QUIK software complex's functionality of online pre-trade operation control.
- The interface supports operations with algo orders of the Algorithmic trading module and OMS orders of the QUIK OMS system.
- The interface does not support REPO operations (except anonymous REPO with CCP).
- Three types of the FIX adapter program interface are used with limits on:
    - The numbers of serviced client accounts (basically, up to 50);
    - The numbers of processed transactions (basically, up to 10 000 per month) – this type of interface is used when the QUIK server is outsourced in the ARQA Technologies data center;
    - The numbers of processed transactions (basically, up to 500 per day) – this interface type is used with the QUIK server installed in the broker's data center.
- Technologically, the Interface is a FIX API to the QUIK server.

1.2 Operating principles

FIX Program Interface provides support of this protocol in QUIK and is intended for connection of external trading systems to the QUIK server for their access to Russian exchanges.

The interface supports FIX version 4.2 and 5.0. If it is necessary the Interface can be adapted to other versions of the FIX protocol. The Interface supports messages for sending orders, reporting on orders and trades passing through the Interface. For detailed information about this protocol visit www.fixtradingcommunity.org.

1.3 Connection scheme



> **Daily restart is necessary for the correct Interface operation. The Interface stop and start must be performed considering the time of session change specified in the ClearingTime setting of the [General] section: the stop must be completed before the time of session change, and the launch must be completed after the time of session change.**

2. Interface installation and configuration

2.1 Technical requirements

2.1.1 For hardware

The FIX Program Interface can be installed on the same computer with the QUIK server or on the other computer connected to the server via local network or the Internet.

Table of equipment and software requirements see on the official website of QUIK.

2.1.2 For communication channels

To connect the FIX Program Interface to the QUIK server via the Internet it is sufficient to have a communication channel capacity of at least 9.6 Kbit/s. The requirements to a dedicated channel used for connection to the foreign partner's network are to be determined by that partner.

2.2 Users to operate with service under Linux

The following users are used to operate and administrate the service:

- Administrator of the platform.

  To denote this user, the root name is used. This user, who knows the root password or is added to the root group, or has access to wide list of commands to execute as the root via the sudo utility.

  This user can install, remove and administrate the service: update, run and stop.

- User, on whose behalf the service is operating.

  To denote this user, the **arqasrv** name is used. When installing the service, the user is created with this name by default.

  This type of user also can administrate the service: update, run and stop – is applied if administration by the root user is unavailable.

Therefore, the service always operates on behalf of the **arqasrv** user. Installation and removing of the service can only be performed by the root user. The service can be administrated by the root or **arqasrv** user.

2.3 Installation

1. Unpack the archive with subdirectories and copy the **fix** directory to the computer where the Interface will be installed.
2. Install the service. The installation procedure depends on the operating system on which the service will run.

   Installation on Windows:

   - Unpack the distributive archive and copy all files from the quikfix/windows sub-directory to the directory where the Interface will be installed.
   - Install the Interface by running the command in the command line:

     ```
     quikfix.exe -install
     ```

     Installation must be done by a user with administrator permissions.

   - Make sure that the service was installed successfully. To do this, open the Windows menu item, type **Services** text in the search and open the corresponding window. Check if the record which identifies the **Quik FIX Adapter Service** is on the list.

   Installation on Astra Linux:

   - The service installation must be done by the root user.
   - The service is installed to the **/opt/arqasrv** directory and on behalf of the **arqasrv** system user by default.
   - Unpack the distributive archive and copy all files from the **quikfix/linux** sub-directory to the directory where current user has permissions to perform operations. For example, home directory: **/home/<user>**.
   - Run the command to install the **jemalloc2** package:

     ```
     sudo apt install libjemalloc2
     ```

   - Add execution permissions to the installation package by running the command:

     ```
     chmod +x install_quikfix-x.x.x.run
     ```

- Install the service by running the command:

```
sudo ./install_quikfix-x.x.x.run -i
```

As a result of executing the installation package, the following actions are performed:

- If there is no **arqasrv** system user, a user with default permissions is created and included in the group of the same name. The **/opt/arqasrv** directory is installed as home directory. The installed service will work on behalf of this user.
- In the **/opt/arqasrv** directory, the **/quik/utilits/quikfix** sub-directory is created. This is the service installation directory into which the distributive files of the installed service will be unpacked.
- The **arqasrv** user is owner of the all files and sub-directories in the service directory.
- A service named **quikfix** will be registered in the **systemd** service management system and run as the **arqasrv** system user.
- A file named **quikfix** is created in the **/etc/arqasrv** directory to fix the service installation parameters.
- The following command files are generated in the service installation directory:
  - **q_start.sh** – start the service including launch parameters. (available service launch parameters are described in 3) Intended to be performed by the arqasrv and root users;
  - **q_stop.sh** – stop the service. Intended to be performed by the arqasrv and root users;
  - **q_status.sh** – display information about service current status. Intended to be performed by the arqasrv and root users;
  - **uninstall.sh** –remove the service. Intended to be performed by the root user;
  - **postmigration.sh** – perform additional operations during migration. Intended to be performed by the arqasrv and root users.

> **If the installation is successfully completed, the message will be displayed:**
> **Installation of service quikfix has been successfully completed.**
> **If the message has a different diagnostic, please contact the QUIK technical support: quiksupport@arqatech.com.**

To be able to use your own (different from default value) name of the service, user – service owner and root directory for service installation, the run file contains launch parameters described in Launch parameters of the Linux installation run package.

- If the installed service will be administrated by the **arqasrv** user, follow the steps:

- To prevent modifications by the arqasrv user in the scripts of the service management, on which the arqasrv user will be granted permission to execute as the root user. Make them immutable by running the following commands:

```
chattr +i /opt/arqasrv/quik/utils/quikfix/q_start.sh
chattr +i /opt/arqasrv/quik/utils/quikfix/q_stop.sh
```

- Give to arqasrv user permissions for executing scripts of the service management by root name, by running the following steps:

  - Run the command:

    ```
    sudo visudo
    ```

  - Add the string in the vi-like editor that opens:

    ```
    arqasrv ALL=(root)
    /opt/arqasrv/quik/utils/quikfix/q_start.sh,/opt/arqasrv/quik/utils/qu
    ikfix/q_stop.sh
    ```

    This string permits the arqasrv user to execute the listed scripts on any computer in the domain under the root name.

The following steps of this instruction can be performed by the arqasrv and root users.

3. Set up the settings in the **quikfix.ini** main configuration file of the Interface:

   - Set up parameters of a FIX connection in a section of [fix_connect_0] view:

     ```
     [fix_connect_0]
     Port=<port number>
     SenderCompID=<identifier of the company that sends a message>
     TargetCompID=<identifier of the company that receives a message>
     ```

   - In the **Classes** setting of the [Quik] section, edit the list of classes by specifying classes to be involved in operations. For example:

     ```
     [Quik]
     Classes=TQBR|TQNE
     ```

4. In the QUIK Administrator program, register a service user on whose behalf the Interface will connect to the QUIK server. In the user settings window do the following:

   - On the Common rights tab, select the API of client's terminal check box.

- Set the user right to use the FIX adapter Program Interface. To do this, on the **Common rights** tab in the **Client modules** table, click the **Edit**… button. In the opened window, select the FIX adapter Program Interface **(Quikfix.exe)** and enable the **Plugin Enabled** permission.
- On the **Classes permissions** tab, set rights to the classes for which you plan to perform operations through the Interface:
- In case of operations with a limited set of clients:
  - Select the **Client** rights template.
  - In the **Firm code** list, select a firm to to perform operations.
  - List client codes separated by comma in the **Client code on the dealer's side** box.
- In case of operations with an unlimited set of clients:
  - Select the **Manager** rights template.
  - In the **Firm code** list, select a firm to perform operations.
  - To register orders on behalf of different firms, select the **Special operations** and **View info for all brokers** check boxes.
  - To manage the session statuses, select the **Special operations** check box.

5. Configure the service user authentication when connecting to the QUIK server.

To do this, contact the QUIK server administrator with a request to receive the pubring.txk and secring.txk keys which will be used to connect to the QUIK server.

- If the QUIK server administrator provided the full set of keys (pubring.txk and secring.txk files), including login and password for them, then:
  - Copy the pubring.txk and secring.txk files to the directory where the Interface is installed.
- If the QUIK server administrator provided the Program to create and manage **KeyGen** keys, then:
  - Use this Program to generate the pubring.txk and secring.txk key pair using the instruction provided by the QUIK server administrator or use the User's manual of the Program to create and manage the **KeyGen*** keys;
  - Copy the pubring.txk and secring.txk files to the directory where the Interface is installed;
  - Send the QUIK server administrator the public part of the pubring.txk key generated in the previous step to register it on the QUIK server. Wait for confirmation of key registration.

> **(*)**
>
> **The pubring.txk public key must contain both the generated public part of the user key and public part of the QUIK server key.**

In the QUIK Administrator, in the **Edit (Users)** dialog box, on the **Security** tab assign the created access keys to the user in the **QUIK keys**.

6. In the Interface directory in the **quikapi.ini** file, set the settings for the user connection to the QUIK server in the [connect] section:

```
[connect]
trade-server-ip-address=<IP address of QUIK server>
port=<client port of the QUIK server>
auth-name=<key of the QUIK server>
```

7. In the **quikfix.ini** main configuration file of the Interface, set up the user name and password in the [QuikAPI] section:

```
[QuikAPI]
User=<user name>
Password=<password>
```

8. Set up automatic daily start and stop of the service according to the following rules:

   - The service must be started after connecting to the QUIK server at least one interface with classes for which the FIX adapter Interface has permissions.
   - The service must be stopped after the end of trading for all classes used by the FIX Program interface.

   If the service is installed on Linux, use the **q_start.sh** and **q_stop.sh** command files located in the directory of installed service to start and stop the service.

9. Run the Interface and make sure that it operates correctly. To do this, run the QMonitor program and check if the information about the successful connection of the service user of the FIX adapter Interface is displayed.

2.3.1 Launch parameters of the Linux installation run package

The installation package supplied in the distributive supports:

-i [<srvname>]
Executing with this launch parameter is available only for the root user.

Installing the service named <srvname>. The <srvname> parameter is optional.
If the<srvname> parameter is missed, the **quikfix** service name is used by default. When a <srvname> parameter is specified, its value will be converted to lowercase, and the service will be installed with a name in lowercase.

Installing two services with the same name is not allowed, even in different root directories.
If you try to do this, the installation will fail with an error.

-d <rootdir>
The launch parameter used when installing the service (with the **-i** launch parameter combination) to specify the root directory for installing the service other than the **/opt/arqasrv** default directory. This is the directory in which the **quik/utils/quikfix** sub-directory is created into which the executable and configuration files of the installed software will be unpacked.

-n <srvusername>
The launch parameter used only when installing the service (with the **-i** launch parameter combination) to specify the <srvusername> user who will be files owner of the service and on whose behalf the installed service will run. The user name by default: **arqasrv**.

-u [<srvname>]
Executing with this launch parameter is available for the arqasrv and root users.

Updating the service named <srvname>. The <srvname> parameter is optional.
If the <srvname> parameter is missed, the **quikfix** service name is used by default.

The service location and the user-owner name of the server are not required, because it is saved in the **/etc/arqasrv/quikfix** directory when installing the service.

-e
Executing with this launch parameter is available for any user.

Unpacking the full archive including configuration file templates. Unpacking is done to a subdirectory with a name equal to the name of the run file (without *.run files), which is created next to the run file in case of absence.

Examples:

Running the run file to install the service:

```
sudo ./install_quikfix-x.x.x.run -i customservice -d /opt/customdir
```

Running the run file to update the service:

```
[sudo] ./install_quikfix-x.x.x.run -u customservice
```

> **Specifying sudo is required if the service is updated by the root user.**

> **The srvname, rootdir, srvusername parameters can only contain values without spaces.**

2.4 Service migration from Windows to Astra Linux

To migrate the Interface service running on Windows OS to the computer with Astra Linux OS, the following is required:

1. Version of the Interface that is migrated from Windows must be equal to the version of the Interface that will be installed on Astra Linux.
2. The migration must be made during strictly non-trading times: after the end of trading and stopping the service, and before the start of trading, before the first launch of the service.

Follow this procedure:

1. Install the Interface service on the computer with Astra Linux.

   Instalattion must be done by the root user.

   - Unpack the distributive archive and copy all files from the **quikfix/linux** sub-directory to the directory where current user has permissions to operations. For example: **/home/<user>**.
   - Run the command to install the jemalloc2 package:

     ```
     sudo apt install libjemalloc2
     ```

   - Run the command to add execution permissions to the run file:

     ```
     chmod +x install_quikfix-x.x.x.run
     ```

   - Install the service by running the command:

     ```
     sudo ./install_quikfix-x.x.x.run -i
     ```

> **If the installation is successfully completed, the following message will be displayed: "Installation of service 'quikfix' has been successfully completed". If another diagnostic is displayed, please contact the QUIK technical support: quiksupport@arqatech.com.**

   - If installed service is to be administrated by the **arqasrv** user, follow the steps:
     - To prevent the **arqasrv** user from modifying service administration scripts for which the user will be granted the root execution permission, make them immutable by running the following commands:

       ```
       chattr +i /opt/arqasrv/quik/utils/quikfix/q_start.sh
       ```

```
chattr +i /opt/arqasrv/quik/utils/quikfix/q_stop.sh
```

- Grant the **arqasrv** user permissions to execute service administration scripts as the root by following the steps:
  - Run the command:

    ```
    sudo visudo
    ```

  - In the opened vi-like editor, add the line:

    ```
    arqasrv ALL=(root)
    /opt/arqasrv/quik/utils/quikfix/q_start.sh,/opt/arqasrv/quik/utils/qu
    ikfix/q_stop.sh
    ```

    This line allows the **arqasrv** user to execute the listed scripts on any computer in the domain as root.

The following steps of this instruction can be performed by the **arqasrv** and **root** users.

2. Copy the following files from the Windows service directory to the Linux service directory:
   - Files: *.ini, *.cfg, *.xml;
   - Key files, paths to which are specified in the qrypto.cfg file;
   - Files, paths to which are addressed by settings in the 4.
3. Execute the following commands in the **/opt/arqasrv/quik/utils/quikfix** directory:

   ```
   chmod +x postmigration.sh
   ./postmigration.sh
   ```

4. Change the settings in the following migrated files:

> **All paths to files and directories must be converted to the form accepted in the Linux file system:**
>
> 1. **The (“\”) backslashes must be replaced with the (“/”) forward slashes.**
> 2. **The names of the directories and files must be converted to lowercase.**

   - [Files] section:
     - LogFileName;
     - SecurityInfo;
     - SecTypes;

- OrderStorage;
- TradeStorage;
- UsTradeStorage;
- OrderHistory;
- FirmsIniDirectory;
- ExtAlgoSpecification;
- SystemInfo.

- [General] section:
  - DigitSignIni.

- [QuikApi*] sections:
  - Ini.

- [fix_connect_*] sections:
  - Storage;
  - Schema;
  - CertFileName;
  - CertPKeyFileName;
  - CertCAFileName.

- [Mail] section:
  - LogFileName.

- Set the passwords in the clear text, if they were encrypted in the Windows service, in the following settings:
  - [QuikApi*] sections:
    - Password.
  - [fix_connect_*] sections:
    - SenderCompID;
    - TargetCompID;
    - CertPass;
    - Password.
  - [Mail] section:
    - Password.

5. Set up automatic daily start and stop of the service. To do this, use the **q_start.sh** and **q_stop.sh** command files located in the directory of the installed service.

2.5 Removing service

Removing the service on Windows and Astra Linux platforms occurs differently.

Removing service installed on Windows

1. Before starting the removal procedure, the service must be stopped.
2. Service removal is performed by a user with administrator permissions.
3. Run the executable file with the **-remove** argument in the directory of the installed service:

```
quikfix.exe -remove
```

As a result of the command execution, the service registration will be removed from the list of services.

The directory where the service is installed is not modified, if required, directory can be deleted manually.

Removing service installed on Astra Linux

Service removal must be performed by the root user.

1. Before starting the removal procedure, the service must be stopped.
2. If the installed service was administrated by the **arqasrv** user, do the following:

   _ Cancel the **arqasrv** user permissions to execute scripts of the service administration as a root by doing the following:

   _Run the command:

```
sudo visudo
```

   _In the opened vi-like editor, delete the line:

```
arqasrv ALL=(root)
/opt/arqasrv/quik/utils/quikfix/q_start.sh,/opt/arqasrv/quik/utils/quikfix/q_sto
p.sh
```

   _ Disable the attribute of immutability of the service administration scripts by running the following commands:

```
chattr -i /opt/arqasrv/quik/utils/quikfix/q_start.sh
chattr -i /opt/arqasrv/quik/utils/quikfix/q_stop.sh
```

3. Run the **uninstall.sh** command file in the **/opt/arqasrv/quik/utils/quikfix** directory of the installed service:

```
sudo ./uninstall.sh
```

As a result of the command execution, the following actions are performed:

_ The **quikfix** service registration is deleted in the systemd service management system.
_ The **/etc/arqasrv/quikfix** file in which the installation service parameters were recorded is deleted.
_ The **/opt/arqasrv/quik/utils/quikfix** directory in which the service is installed, changes the name to the **/opt/arqasrv/quik/utils/quikfix.backup YYYY.MM.DD_hh.mm.ss**.

> **Renaming, rather than deleting, a directory is done in case the data from it is required. If there is no such need, then delete this directory manually.**

> **If the service is successfully removed, the following message will be displayed: "Removing of service 'quikfix' has been successfully completed".**
> **If another diagnostic is displayed, please contact the QUIK technical support: quiksupport@arqatech.com.**

2.6 quikfix.ini file format

> **For the correct Module operation, the configuration file must have the Windows-1251 encoding.**

2.6.1 [Files] section

- **LogFileName** – path to the log file. Value by default: logs/quikfix.log.
- **LogDepth** – depth of log file rotation.
- **OrderStorage** – path to the file with orders. Value by default: import/orders.data.
- **TradeStorage** – path to the file with trades. Value by default: import/trades.data.
- **UsTradeStorage** – path to the file with trades for execution. Value by default: import/ustrades.data.
- **OrderHistory** – path to the file with order history. Value by default: import/orderhistory.data.

> **If you delete file with order history then information about active GTC and GTD orders entered via the Interface will be lost.**

- **PSFile** – path to the file with encrypted setting values. If the path is not specified then encrypted setting values are stored in quikfix.ini. Relative path cannot be used. Value by default is not specified. Setting values can be encrypted if necessary. For details, see 4.

- **FirmsIniDirectory** – path to directory where the Interface saves the configuration files of the Dealer Library. Value by default: firmsini.
- **SecurityInfo** – name xml file with correspondences between instruments and their alternative codes. Value by default: secinfo.xml. File format is described in 2.10.
- **SecTypes** – name and path to the file with configured correspondences between instrument types and value transmitted in tag [167] SecurityType. Value by default: sectypes.xml. File format is described in 2.155.
- **SystemInfo** – path to xml-file to store parameters of FIX sessions. Value by default: data/systeminfo.xml.
  File format:

```
<SystemInfo SaveTime="YYYYMMDD-HH:MM:SS">
  <Block Name="FIXSESSION:SenderCompID_TargetCompID">
    <SeqNoGroup>
      <Inbound>Expected value of incoming messages counter</Inbound>
      <Outbound>Following value of outgoing messages counter</Outbound>
    </SeqNoGroup>
  </Block>
</SystemInfo>
```

> **Counter values of outgoing and incoming messages can be changed in file only in case of emergency situation when the receiving party (FIX client) cannot reset their counters of messages for any reason and expects to receive a certain value of counter from the Interface. If you are not sure you need to change the counters, please contact the QUIK technical support: quiksupport@arqatech.com.**

- **ExtAlgoSpecification** – path to file with specifications of external algo order format. Value by default: desc/ea.xml.

> **It is not recommended that you edit the ea.xml.**

- **ErrMsgUrgency** – mask of error message severity levels saved in log file. The mask can consist of any combination of HIGH, NORMAL and LOW values separated by "|". Value by default: HIGH|NORMAL|LOW.
- **WrnMsgUrgency** – mask of warning severity levels saved in log file. The mask can consist of any combination of HIGH, NORMAL and LOW values separated by "|". Value by default: HIGH|NORMAL|LOW.
- **InfoMsgUrgency** – mask of information message severity levels saved in log file. The mask can consist of any combination of HIGH, NORMAL and LOW values separated by "|". Value by default: HIGH|NORMAL|LOW.

- **DebugMsgUrgency** – mask of debugging message severity levels saved in log file. The mask can consist of any combination of HIGH, NORMAL and LOW values separated by "|". Value by default: HIGH|NORMAL|LOW.

  For example:

```
[Files]
LogFileName=logs/quikfix.log
OrderStorage=import/orders.data
TradeStorage=import/trades.data
UsTradeStorage=import/ustrades.data
OrderHistory=import/orderhistory.data
PSFile=import/psfile.data
FirmsIniDirectory=firmsini
SecurityInfo=secinfo.xml
SecTypes=sectypes.xml
SystemInfo=data/systeminfo.xml
ExtAlgoSpecification=desc/ea.xml
ErrMsgUrgency=HIGH|NORMAL|LOW
WrnMsgUrgency=HIGH|NORMAL|LOW
InfoMsgUrgency=HIGH|NORMAL|LOW
DebugMsgUrgency=HIGH|NORMAL|LOW
```

2.6.2 [QuikAPI] section

This section is intended for identification of connection to the QUIK server:

- **User** – user name for connection to the QUIK server.
- **Password** – password for connection to the QUIK server. A password must not begin with "$" and "#" characters. A password can be encrypted if necessary (for details, see 4).
- **Language_ID** – system language identifier for sending to QAPI. Valid values:
  - 9 (by default) – English;
  - 25 – Russian.
- **Ini** – name of file with settings for connection to the QUIK server. Value by default: quikapi.ini. For more details about settings in the file, see 2.7.
- **LogDir** – path to the directory that contains the log files of the QuikApi library. Value by default: <current folder>.
- **DatDir** – path to the directory that contains the files for storing the QuikApi library data. Value by default: <current folder>.
- **ParamTimeout** – period of receiving the updated current parameters of the instruments by Interface, in seconds. The setting can be used for scaling of updates flow of market data when requesting the Quotes table data by the Interface. Value by default: 0.
- **SwitchToConnID** – identifier of the fallback QuikApi connection to which the Interface connects automatically (after the last attempt of connection to the QUIK server occurred)

when the time interval specified in the ActiveQuikApiConnSwitchThreshold setting expires. Value by default: 0 (not specified).
To automatically reconnect to [QuikApi] section, specify SwitchToConnID=DefConnID.

For example:

```
[QuikAPI]
User=FixAdapter_01
Password=$111111
Language_ID=9
Ini=QuikApi.ini
ParamTimeout=3
```

2.6.3 [QuikAPI_<ConnID>] section

The section of [QuikApi_<ConnID>] view, where <ConnID> is an identifier of connection, specified in the ActiveQuikApiConnId and / or SwitchToConnID parameter, is intended for setting up parameters of a fallback connection to another QUIK server without relaunch of the Interface. The settings in sections of [QuikApi_<ConnID>] view are similar to [QuikApi] section.
Example of cyclic reconnection among three connections:

```
[General]
ActiveQuikApiConnID=1
ActiveQuikApiConnSwitchTimeout=10

[QuikApi_1]
User=FIX
Password=qwe
Ini=QuikApi_1.ini
SwitchToConnID=2

[QuikApi_2]
User=FIX
Password=qwe
Ini=QuikApi_2.ini
SwitchToConnID=3

[QuikApi_3]
User=FIX
Password=qwe
Ini=QuikApi_3.ini
SwitchToConnID=1
```

2.6.4 [fix_connect_<ConnectionID>] section

Sections of [fix_connect_<ConnectionID>] view, where <ConnectionID> is an arbitrary identifier of connection, are intended for setting connection parameters.

- **ApplVerID** – protocol version which is used in current session. Valid values:
  - FIX42 (by default);
  - FIX44;
  - FIX50;
  - FIX50SP2.
- **Type** – connection type:
  - IN – incoming;
  - OUT (by default) – outgoing.
- **UserName** – user name to be specified in UserRequest message if protocol FIX 5.0 is used.
- **Password** – user password to be specified in UserRequest message if protocol FIX 5.0 is used. A password must not begin with "$" and "#" characters. The setting value can be encrypted if necessary (for details, see 4).
- **Port** – port number. Value by default: 8000.
- **Port2** – number of fallback port (for OUT connections). Value by default: 8001.
- **IP** – depends on the type of connection:
  - for **OUT** connections – client IP-address;
  - for **IN** connections – the IP address of the local network interface in which the Interface will wait for a client connection. If the value is not specified and there is more than one network interface on the computer, then the client connection will be accepted on any of them.
- **IP2** – fallback IP of the client (for OUT connections). Value by default: "" (empty).
- **SenderCompID** – identifier of company, which sent message. In pair with 'TargetCompID' key forms session identifier. A password must not begin with "$" and "#" characters. The setting value can be encrypted if necessary (for details, see 4).
- **TargetCompID** – identifier of company, which receives message. In pair with 'SenderCompID' key forms session identifier. A password must not begin with "$" and "#" characters. The setting value can be encrypted if necessary (for details, see 4).

> **If two connections are set up then values of SenderCompID and TargetCompID must be unique for each connection.**

- **DeliverToCompID** – additional identifier intended for filling tag [128] DeliverToCompID.
- **OnBehalfOfCompID** – additional identifier intended for filling tag [115] OnBehalfOfCompID.
- **Storage** – path to the storage file of the Interface's messages (separate file for each connection). Value by default: import/storage.data.

- **Schema** – path to the file with scheme, which is applied to the messages sent via the current connection. Value by default: "" (empty).
- **Timeout** – timeout in seconds of establishing connection. Value by default: 10.
- **Keepalive** – interval in seconds of sending heartbeats. Value by default: 30.
- **Timeshift** – time shift between the Interface and QUIK server in <HHMMSS> format.
    - if SendTradeTimeGMT setting is activated, then the shift is added to the server time. After that the total value is transferred to GMT;
    - if SendTradeTimeGMT setting is deactivated, then the shift is taken off the local time and all the time parameters are transferred to the QUIK server time and transmitted to FIX client.
- **IsPrimary** – attribute which defines if the current connection is main or additional. Valid values:
    - 0 (by default) – additional. Sending Execution Report, Cancel Reject messages is blocked for additional connections.
    - 1 – main connection.
    - For additional sessions processing of the following client messages is blocked:
        - New Order Single;
        - Order Cancel Request;
        - Order Cancel Replace Request;
        - Order Status Request.
- At the current moment only two FIX connections can be created. For one of them IsPrimary key must be equal to 1, and for another – 0.
- **TradeReportingStyle** – type of messages which are generated by trades. Valid values:
    - 0 (by default) – ExecutionReport;
    - 1 – TradeCaptureReport;
    - 2 – ExecutionReport + TradeCaptureReport.
- **TcpNoDelay** – attribute of using Nagle's algorithm for data transmission. Valid values:
    - 0 – Nagle's algorithm is used;
    - 1 (by default) – is not used.
- **MaxFailureConnectionAttempts** – maximum number of connection attempts over a cycle. The fallback IP address and port are used if connection failed after reaching the specified number of connection attempts. The parameter of OUT connections. Value by default: 0 (fallback IP address and port are not needed).
- **ClientReadyTest** – indicates that a FIX client is checked for readiness to receive the data. The check is performed by sending TestRequest after the session is established and waiting for response. Valid values:
    - 0 – check;
    - 1 (by default) – do not check.
- **TextEncoding** – encoding of outgoing FIX messages. Valid values:
    - ANSI (by default);

- UTF-8.

Secure connection settings

- **UseSecureConnection** – indicates that a secure connection (TLS/SSL) is used. Valid values:
  - 0 (by default) – unsecure connection is used;
  - 1 – SSL connection is used.
- **CertFileName** – path and a name of the file of the connection certificate and optional private key (in PEM format). It is used if UseSecureConnection=1. Value by default: "" (empty).
- **CertPKeyFileName** – path and a name of the file of the private key (in PEM format). If this setting is missing, it is considered that the key is specified in the file set by the **CertFileName** setting. It is used if UseSecureConnection=1. Value by default: "" (empty).
- **CertPass** – password of an SSL certificate. Parameter is used if a secure connection is used. It is used if UseSecureConnection=1. A password must not begin with "$" and "#" characters. The parameter of 'OUT' connections. Value by default: "" (empty). The setting value can be encrypted if necessary (for details, see 4).
- **VerifyPeerCertificate** – depends on the type of connection:
- For **OUT** connections – indicates that the server certificate signature will be verified with the root certificate. Valid values:
  - 0 (by default) – do not verify the server certificate signature;
  - 1 – verify the server certificate signature with the root certificate.
- For **IN** connections – indicates that the client's certificate will be requested and its signature will be verified with the root certificate. Valid values:
  - 0 (by default) – do not request the client's certificate;
  - 1 – request the client's certificate and verify its signature with the root certificate.
- **NetTrace** – indicates that network connection trace is used. Valid values:
  - 0 (by default) – not used;
  - 1 – used.
- **CertCAFileName** – name and path to the container file of certification authority's root certificates. It is used if UseSecureConnection=1. Value by default: "" (empty).
- **SSLMethod** – type and version of SSL/TLS protocol. Valid values:
  - TLSv1.0 (by default);
  - TLSv1.1;
  - TLSv1.2;
  - SSLv3.0.
- **CertNameWhiteList** – list of certificate names separated by commas for which an incoming TLS connection is allowed. If the list is not set or empty, a certificate name is not checked. Value by default: "" (empty). It is used for the following settings values:
  - UseSecureConnection=1;

- Type=IN;
- VerifyPeerCertificate=1.

- **CertSerialNumberWhiteList** – list of certificate serial numbers separated by commas for which an incoming TLS connection is allowed. If the list is not set or empty, a certificate serial number is not checked. Value by default: "" (empty). It is used for the following settings values:
  - UseSecureConnection=1;
  - Type=IN;
  - VerifyPeerCertificate=1.

  For example:

```
[fix_connect_1]
ApplVerID=FIX42
Type=OUT
port=15840
port2=15841
ip=122.1.1.1
ip2=122.1.1.2
SenderCompID=RAF
TargetCompID=MOS
DeliverToCompID=DTID
OnBehalfOfCompID=BOID
Storage=import/storage1.data
Storage=rules/default30.xml
timeout=10
keepalive=1800
timeshift=60000
IsPrimary=1
TradeReportingStyle=1
TcpNoDelay=1
MaxFailureConnectionAttempts=3
ClientReadyTest=0
UseSecureConnection=1
CertFile=fxgo_cert_key.pem
CertPass=12345
NetTrace=1
CAFile=fxgo_ca.pem
SSLMethod=TLSv1.2
```

> **If the Interface uses more than one FIX connection then paths to the storage files should be clearly set up and shouldn't be the same with values by default. Names of storage files must be unique.**

Data preparation example

To prepare data for configuration with incoming TLS connection with the openssl utility used (key name is case-sensitive), follow the steps below:

4. Create a private key for a root certificate:

```
openssl genrsa -out rootca.key 2048
```

5. Create a root certificate with validity period of 365 days:

```
openssl req -x509 -new -nodes -key rootca.key -days 365 -out rootca.cer
```

6. Create a server certificate signed by a root certificate as follows:

   _ Create a private key for a server certificate:

   ```
   openssl genrsa -out server.key 2048
   ```

   _ Create a request to sign the server certificate:

   ```
   openssl req -new -key server.key -out server.csr
   ```

> **The value of the Common Name parameter specified when creating a request must differ from the Common Name of the root certificate.**

   _ Sign the server certificate request by the root certificate:

   ```
   openssl x509 -req -in server.csr -CA rootca.cer -CAkey rootca.key -
   CAcreateserial -out server.cer -days 365
   ```

7. Specify the path to the server.cer file in the **CertFileName** setting of section [fix_connect], and specify the path to the server.key file in the **CertPKeyFileName** setting of section [fix_connect].
8. If the server certificate validity check is enabled at the client's side, specify the rootca.cer file as a container file for root certificates (CA) in the client application settings.

   Example of anonymous connection settings:

```
[fix_connect_pt]
```

```
...
Type=IN
UseSecureConnection=1
NetTrace=1
SSLMethod=TLSv1.1
CertFileName=./TLS/server.cer
CertPKeyFileName=./TLS/server.key
```

9. If non-anonymous connection is configured, create a certificate and a private key for a client, and then sign it by the root certificate:

- Create a private key for a client's certificate:

```
openssl genrsa -out client.key 2048
```

- Create a signature request for the client's certificate:

```
openssl req -new -key client.key -out client.csr
```

- Sign the client's certificate request by the root certificate:

```
openssl x509 -req -in client.csr -CA rootca.cer -CAkey rootca.key -
CAcreateserial -out client.cer -days 365
```

Example of non-anonymous connection settings:

```
[fix_connect_pt]
...
Type=IN
UseSecureConnection=1
NetTrace=1
SSLMethod=TLSv1.1
CertFileName=./TLS/server.cer
CertPKeyFileName=./TLS/server.key
CertCAFileName=./TLS/rootca.cer
VerifyPeerCertificate=1
```

Created files of client's certificate and key must be specified in the client application settings.

2.6.5 [WhiteIPs] section

[WhiteIPs] section contents the list of IP-addresses, connections from which will be accepted by the Interface. If the section is absent or empty then connections from any addresses will be accepted.

This option is applied only for **IN** connections.

For example:

```
[WhiteIps]
192.168.32.100
192.168.32.101
```

2.6.6 [service] section

- **displayname** – displayed service name.
- **name** – system service name.

  For example:

```
[service]
displayname=QuikFixService
name=QuikFixService
```

2.6.7 [Quik] section

- **Classes** – list of instrument classes separated by a vertical bar ("|") on which the FIX client is allowed to request the market data and send / cancel / replace orders. The "*" templates to set a filter by mask are allowed to be used, for example: "TQ*". If the "*" value is set, all classes are used.

  > **This setting does not affect the number of classes and instruments that are available for the Interface and given to the client by the Security Definition Request message.**

- **Clientcode** – client code by default from the name of which trading will be held. The parameter is used if the value is not typed in transaction,
- **Account** – trade account by default from which trading will be held. The parameter is used if the value is not typed in transaction.

  For example:

```
[Quik]
Classes=TQBR|TQNE
```

```
Clientcode=C1
Account=0001
```

2.6.8 [CodeChanges] section

The section allows setting the correspondences between the client codes in the QUIK system and the client codes transmitted in tag [109] of New Order Single and Order Cancel Replace Request messages by the FIX client and if necessary. When entering transactions to the server the client code specified by the client in the FIX message is substituted by the client code in the QUIK system specified in this section. When reporting orders and trades to a client the back-substitution is carried out. This setting is available only for the spot markets.
The correspondences are set up as follows:

```
<Client code in FIX>=<Client code in QUIK>
```

For example:

```
[CodeChanges]
TEST=B9
```

2.6.9 [General] section

- **Comment** – comment which is added to the client code automatically on order entering (if CommentType=0).

  For example:

  ```
  Comment=ITRADE
  ```

  Where:

  - ITRADE is a comment which will be added to all orders to be sent via Interface.
- **CommentType** – defines the method of filling comment on order entering. Value by default: 0. Valid values:

CommentType	Condition	Value of Comment field
0	CodeToAccount≠2	Value of setting Comment
	CodeToAccount=2	Value of setting Comment + '/' + value of tag [109] ClientID Separator '/' is added if value of setting Comment is selected
1		Value of tag [11] ClOrdID

CommentType	Condition	Value of Comment field
2		MOID label of order. Value is formed in accordance with settings in section [MemoOrderID], see 2.6.24
3		Value of tag [58] Text
4		Value of tag [1] Account + '#' + value of tag [109] ClientID
5		Value of tag [58] Text + value of tag specified in the MemoSfxTag setting. To encode the value of tag from the MemoSfxTag setting enable the MemoSfxEncoding setting

The resulted comment is combined with client code (resulted in accordance with CodeToAccount setting) and transmitted to server in BROKERREF transactional field. Client code and comment are defined to be separated by one or two "/" characters depending on the configuration of Limits Calculation Library setting Client code may include "/".

- **MemoSfxTag** – number of tag which contains a comment suffix. The value must be positive integer. It is used when CommentType=5.
- **MemoSfxEncoding** – attribute of encoding the value of tag specified in the **MemoSfxTag** setting before adding it to the order comment. Valid values:
  - 0 – encoding is disabled;
  - 1 (by default) – enable encoding in Base-62.
- **MassCancelByDisconnect** – cancel active orders when breaking a FIX session. Cancellation request is sent to server for each client order with Active status immediately at the moment of the session break. Valid values:
  - 0 (by default) – do not cancel active orders;
  - 1 – cancel active orders. At the following connection the client will receive the ExecutionReport message on all cancelled orders with Cancelled status if ClearingTime did not pass.
- **CFAEnable** – attribute of client code and trade account transmission in the Account field. Valid values:
  - 0 (by default) – client code and trade account are transmitted separately in corresponding fields;
  - 1 – client code and trade account are transmitted in the Account field and are separated by the sign from CFASeparator key.

For example:

```
CFAEnable=1
```

- **CFASeparator** – sign which separates client code from trading account if value of CFAEnable is 1. Value by default: :DEP:.

  For example:

```
CFASeparator=:DEP:
```

- **CodeToAccount** – selection of the mode of client codes and trade accounts substitution on transmission to the QUIK server. Valid values:
  - 0 (by default) – substitution is deactivated;
  - 1 – client code is taken from tag [109], trading account is selected on the bases of correspondence to the value of tag [109], which is specified in the 2.6.20 section;
  - 2 – client code is taken from tag [1], trading account is selected on the bases of correspondence to the value of tag [1], which is specified in the 2.6.20 section;
  - 3 – client code is taken from tag [109], trading account is selected on the bases of correspondence to the value of tag [1], which is specified in the 2.6.20 section;
  - 4 – trading account is taken from tag [1], client code is selected on the bases of correspondence to the value of tag [448], which is specified in the 2.6.20 section;
  - 5 – trading account is taken from tag [1], client code is selected on the bases of correspondence to the value of tag [1], which is specified in the 2.6.20 section;
  - 6 – trading account is selected on the bases of value of Partner field of a trade and of correspondence specified for this value in the 2.6.20 section;
  - 7 – trading account is taken from the account setting in the [Quik] section, client code is from the clientcode setting in the [Quik] section. Values specified by the client in tags Account1 and ClientID109 are ignored but transmitted back to the client in execution reports.

> **The resulted client code is combined with client code (resulted in accordance with CommentType setting) and transmitted to server in BROKERREF transactional field. Client code and comment are defined to be separated by one or two "/" characters depending on the configuration of Limits Calculation Library setting Client code may include "/".**

For example:

```
CodeToAccount=0
```

- **OB_EnableLevel2** – attribute of Level II quotes table transmission. Valid values:
  - 0 – Level II quotes table transmission is deactivated;
  - 1 (by default) – is activated.

  Settings from [market <Market code>] and [class <Class code>] sections have the higher priority.
  For example:

```
OB_EnableLevel2=1
```

- **OB_MarketDepth** – quantity of the best quotes. Value by default: 5. Zero value is for displaying all the quotes. Settings from [market <Market code>] and [class <Class code>] sections have the higher priority.
  For example:

```
OB_MarketDepth=15
```

- **SendTradeTimeGMT** – attribute of date and time conversion to GMT format in orders and trades. Valid values:
  - 0 (by default) – conversion is deactivated;
  - 1 – activated.

  For example:

```
SendTradeTimeGMT=0
```

- **UseClassForExDest** – method of filling tag [100] ExDestination in execution reports. Valid values:
  - 0 (by default) – the tag has exchange code;
  - 1 – class code.

  For example:

```
UseClassForExDest=0
```

- **HeaderTagsFromLogon** – method of transmitting tags [115] OnBehalfOfCompID and [128] DeliverToCompID. Valid values:
  - 0 – reply for transaction will content these tags only if the transaction contents them;
  - 1 (by default) – the tags will be transmitted in all the messages if Logon message contents these tags.

For example:

```
HeaderTagsFromLogon=1
```

- **LotsQty** – method of displaying security quantity. Valid values:
  - 0 (by default) – in items;
  - 1 – in lots.

  The setting has the higher priority relative to the analogical settings from [market_<Market code>] and [class_<Class code>] sections.

> **Quantity in OMS orders is transmitted in items regardless of the value in the LotsQty parameter.**

For example:

```
LotsQty=0
```

- **MarketData** – attribute of subscription for market data. Valid values:
  - 0 – deactivated;
  - 1 (by default) – subscription is activated.

For example:

```
MarketData=0
```

- **ActiveQuikApiConnId** – identifier of the QuikApi connection, which is intended for reconnection of FIX Adapter Program interface to another QUIK server without restart. Connection settings are to be set up in section of [QuikApi_<ConnID>] view.
  For example:

```
ActiveQuikApiConnId=10154
```

- **ActiveQuikApiConnSwitchTimeout** – time period since the moment at which the connection is broken down after which the Interface stops trying to connect to the current QUIK server and reconnects automatically to a fallback connection specified, in seconds. Value by default: 0 (not specified).

> **If the value of ActiveQuikApiConnSwitchTimeout parameter is not specified, the Interface will not reconnect to a fallback connection, specified in the SwitchToConnID setting of [QuikApi_<ConnID>] section.**

- **IgnoreReplaceQtyCheck** – disable quantity value control on order replacement. Valid values:
  - 0 (by default) – quantity value control is enabled;
  - 1 – control is disabled.

  The option is applied only to active or partially filled orders.
  For example:

```
IgnoreReplaceQtyCheck=1
```

- **DisableOrderReopen** – disable replacement of executed order (with the help of tag [35]=G). Valid values:
  - 0 (by default) – replacement of executed order is allowed;
  - 1 – replacement is prohibited.

  For example:

```
DisableOrderReopen=1
```

- **ClearingTime** – time of session change in format <HHMMSS>. The fact of session change is defined on Interface's startup by comparing this value, a local time of the last FIX session's close (StopTime) and surrent local time (StartTime). It is accepted that session change occurred if any of these conditions are true:
  - current date differs from the stop date by more than a day;
  - stop occurred yesterday (ClearingTime < StartTime);
  - stop occurred today (StopTime < ClearingTime < StartTime).

  When the session is changed, the storage file of Interface messages will be cleared and counters (ExecID, TransID, RN) will be reset to 1. Value by default: 0.
  For example:

```
ClearingTime=030000
```

- **FailedReplaceReportingMode** – activation of reporting mode of deleted order in cases when a new order cannot be sent while order replace. Valid values:
  - 0 (by default) – ExecutionReport message with 'Canceled' status will content order identifier in tag [41] OrigClOrdID and replace identifier in tag [11] ClOrdID received from order replace request;
  - 1 – ExecutionReport message with 'Canceled' status will content only order identifier in tag [11] ClOrdID.

For example:

```
FailedReplaceReportingMode=0
```

- **AllowReplace** – set up allowance of order replace operations. Valid values:
  - 0 – order replace is unavailable;
  - 1 (by default) – is available.

For example:

```
AllowReplace=1
```

- **SymbolSource** – defines filling tag [55] Symbol on Execution Report transmission. Valid values:
  - 0 (by default) – instrument short name is transmitted;
  - 1 – instrument full name;
  - 2 – instrument code;
  - 3 – ISIN code.

For example:

```
SymbolSource=0
```

- **ClientSymbolIDSource** – identifier type transmitted in tag [55] Symbol if ClientSecurityIDType=1. Valid values:
  - 1 – CUSIP;
  - 2 – SEDOL;
  - 4 – ISIN;
  - 5 – RIC;
  - 6 – ISO;
  - 8 (by default) – instrument code in QUIK;
  - 100 – instrument short name in QUIK;
  - 101 – instrument registration number.

For example:

```
ClientSymbolIDSource=6
```

- **ClientSecurityIDType** – set up the method of instrument identification in incoming messages. Valid values:

- 0 (by default) – tag [48] SecurityID has instrument code, short name, ISO or ISIN code (depending on the value of tag [22] IDSource), value of tag [55] Symbol is ignored;
- 1 – tag [55] Symbol has instrument code, value of tag [22] IDSource is supposed to be equal to 8 (instrument code), value of tag [48] SecurityID is ignored.

For example:

```
ClientSecurityIDType=1
```

- **OrdTypeTag** – number of alternative tags for order type in New Single Order message. If the tag has a non-null value then this value will be used to define order type. If the option is set up but New Single Order message does not have alternative tag with the selected number, then value of the standard [40] OrdType tag will be used. Value by default: 0.
For example:

```
OrdTypeTag=6000
```

- **TreatQtyAsNominalValueForBonds** – indicates that the value of [38] OrderQty tag will be interpreted as a total face value in requests for order for obligation entering and replacement. On orders and trades with obligation reporting all the quantity parameters of ExReport message ([38] OrderQty, [32] LastQty, [14] CumQty, [151] LeavesQty) will also be calculated with account of face value. Valid values:
  - 0 (by default) – option is disabled;
  - 1 – option is enabled.

For example:

```
TreatQtyAsNominalValueForBonds=0
```

- **GenOTCTrade*** – attribute which shows that trades are generated using the target currency. Valid values:
  - 0 – trades generation using the target currency is deactivated. In this case tag [119] SettlCurrAmt has 0.000000 value and for trades in target currency session status is not checked. The settings which set up in [ConvSettings] section are ignored;
  - 1 (by default) – trades registration using the target currency is activated.

For example:

```
GenOTCTrade=0
```

- **FxSecurityCheckFilter*** – list of classes separated by '|' for which availability of the session and its status is controlled before the transaction of an order sending. Valid values:
  - CV (by default) – instrument is in target currency;
  - FX – conversional instrument.
- **CheckHandlInst** – attribute of checking if tag [21] exists in New Order Single and Order Cancel Replace Request messages. Valid values:
  - 0 – checking is disabled;
  - 1 (by default) – checking is enabled.

  For example:

```
CheckHandlInst =1
```

- **AlgoTimeFormat** – format of time parameters in algo orders. Valid values depending on the message status:
  - 0 (by default) – QUIK server time in <HHMMSS> format;
  - 1 – GMT in <HHMMSS> format. Type in the FIX protocol is UTCTimeOnly.

  For example:

```
AlgoTimeFormat=0
```

- **TransactTimeFillType** – method of filling tag [60] TransactTime in Execution Report messages. Valid values:
  - 0 (by default) – tag [60] TransactTime in Execution Report messages is filled by the following value depending on the message status (tag [150]):

Message status	Status value	Description
0	New	Date and time of order registration from the Interface or, if that value is not available, the date and time when the transaction is received from the Interface by the QUIK server
1	Partially Fill	Time of trade execution
2	Fill	Time of trade execution
3	Done For Day	Time of order registration
4	Canceled	Time of order registration
5	Replaced	Time of order registration

Message status	Status value	Description
6	Pending Cancel	Time of order registration
8	Reject	Time of order registration
E	Pending Replace	Time of order registration
A	Pending New	Time of order registration
C	Expired (for OMS orders)	Time of order registration
D	Restated (for OMS orders)	Time of order registration
L	Triggered or Activated by System	Time of order registration

– 1 – tag [60] TransactTime in Execution Report messages is filled by the following value depending on the message status (tag [150]):

Message status	Status value	Description
0	New	Date and time of order registration from the Interface or, if that value is not available, the date and time when the transaction is received from the Interface by the QUIK server
1	Partially Fill	Date and time of trade execution
2	Fill	Date and time of trade execution
3	Done For Day	Time of message formation (time on the computer where the Interface is installed)
4	Canceled	Time of order cancellation
5	Replaced	Time of receiving response to the transaction of new order registration by the QUIK server (with a zero value of fractions of a second)
6	Pending Cancel	Time of message formation (time on the computer where the Interface is installed)
8	Reject	Time of order registration
E	Pending Replace	Time of message formation (time on the computer where the Interface is installed)

Message status	Status value	Description
A	Pending New	Time of message formation (time on the computer where the Interface is installed)
C	Expired (for OMS orders)	Time of message formation (time on the computer where the Interface is installed)
D	Restated (for OMS orders)	Time of message formation (time on the computer where the Interface is installed)
L	Triggered or Activated by System	Time of message formation (time on the computer where the Interface is installed)

For example:

```
TransactTimeFillType=1
```

- **CommentNOFX*** – prefix in the beginning of the comment of the client's trade, which means that this trade should not be converted. Value by default: "" (empty).
  For example:

```
CommentNOFX=nofx
```

- **CommentDOFX*** – prefix in the beginning of the comment of the client's trade, which means that this trade is to be converted. Value by default: "" (empty).
  For example:

```
CommentDOFX=dofx
```

> **CommentNOFX and CommentDOFX settings are mutually exclusive. Specifying / not specifying both settings is not allowed.**

> **(*) The options are used if the trades are reported via the Interface which passed the currency conversion with the help of FX Convertor module. The values of the options must coincide with the same options in FxConverter.ini of FX Convertor module.**

- **EnableLotSizeChecking** – attribute of lot size control transmitted in tag [6200] of NewOrderSingleD and OrderCancelReplaceRequest client messages for correspondence to actual lot size of the instrument in QUIK. Valid values:
  - 0 (by default) – control is disabled;
  - 1 – control is enabled.

  For example:

```
EnableLotSizeChecking=1
```

- **ReOffer** – attribute of approval to any rate in the tag [5092] of the NewOrderSingleD client message when submitting orders for offering. Valid values:
  - N – no;
  - Y – yes.
- **EnableConversionFlagReport** – attribute of conversion mode of in tag [6201] CheckLotFlag of ExecutionReport message. Valid values:
  - 0 (by default) – transmitting is disabled;
  - 1 – transmitting is enabled.

  For example:

```
EnableConversionFlagReport=1
```

- **LastPxSource** – source of the last trade price when transmitting market data. Valid values:
  - 0 (by default) – Quotes table parameter;
  - 1 – all trades flow.

  For example:

```
LastPxSource=1
```

- **CheckSettleDate** – attribute of checking for matching between settlement date specified by client in tag [64] SettlDate with settlement date of the instrument. Valid values:
  - 0 (by default) – disable check;
  - 1 – enable check.

> **The same name settings in section [class_<class code>] (see 2.6.19) and [market_<market code>] (see 2.6.18) have higher priority than settings of this section.**

- **ClientIDValidityTokenTag** – number of the tag containing the validation key for a client code according to the [ClientIDValidityTokens] section. Value by default: 0 (not specified).
- **TimeStampFormat** – format of the time fields in outgoing FIX messages. Valid values:
  - NOFRAC – without fraction of a second (in the <YYYYMMDD-HH:MM:SS> format);
  - MS – with milliseconds (in the <YYYYMMDD-HH:MM:SS:mmm> format);
  - MCS – with microseconds (in the <YYYYMMDD-HH:MM:SS:uuuuuu> format).
- **UseOrigClientCodeForCrossTrades** – attribute of clients original code display in the reports on cross trades, which were concluded after the client code change. Valid values:
  - 0 (by default) – use the trade client code;
  - 1 – use the client's order original code.
- **KeepOrderParties** – transmitting of the original 'Parties' group from the client's request for submitting of an order in all 'Execution Report' messages for this order. Valid values:
  - 0 – the original group is not saved, OrderOriginationFirm and OrderOriginationTrader elements (if values are available) are transmitted in Execution Report messages for orders and trades in the group;
  - 1 – the original group from the client's request is transmitted in all 'Execution Report' messages.
- **WaitSecurityDescTimeOut** – timeout (in seconds) while waiting for the instrument description according to request. Value by default: 10.
- **StrictCurrencyCheck** – attribute of control of compliance between the price currency, specified by the client in tag [15] Currency, and trade currency of instrument when entering/replacing an order. Possible values:
  - 0 (by default) – control is not executed;
  - 1 – control is executed.

  If currencies are different, the search of currency correspondences in [CentPresentation] section is carried out. If correspondences are not found, client request is rejected by the Interface.
- **AllowDynamicSecurityCreation** – indicates that the Interface sends a request for the new instrument creation to the gate, if the instrument specified by a client is not found in the gate's dictionaries. Valid values:
  - 0 (by default) – the request is not sent;
  - 1 – the request is sent.

  This functionality is available, if the gate supports the transaction of new instrument creation.
- **MergePrevAndClosePrices** – indicates that prevprice and closeprice parameters of the Quotes table are combined when transmitting the MDEntryType parameter equal to 5 in Market Data Refresh message. Valid values:
  - 0 (by default) – prevprice is only transmitted;
  - 1 – prevprice and closeprice parameters are combined when transmitting. Updating any of the parameters generates Market Data Refresh for MDEntryType=5.

- **OnBehalfOfUIDTag** – number of tag which contains the user identifier on whose behalf the order is entered / cancelled. The tag value is transmitted to a transaction of order entry / cancellation if a corresponding field is supported in this transaction. Value by default: 0 (not specified).
- **WaitCancelledOrderByReplace** – indicates that the Interface waits for an order that is replaced in the Cancelled status before placing a new one. Valid values:
  - 0 (by default) – disabled;
  - 1 – enabled.
- **OrderHistoryRetentionPeriod** – retention period of multi-day order history after the last order is received from server, in days. Value by default: 30. Minimal valid value: 30.
- **SecurityDefinitionBatchSize** – the number of instruments in one message when sending the instrument dictionary to the client. To disable sending the dictionary in parts (sending the entire dictionary in one message), set the value equal to 0. Value by default: 100.
- **SendSnaphotForIncremenalMDSubscription** – indicates that an initial snapshot (Market Data – Snap Shot / Full Refresh messages) is sent before transmitting the updates (Market Data Incremental Refresh messages) when registering the subscription on market data of the Incremental Refresh type. Valid values:
  - 0 (by default) – do not send a snapshot;
  - 1 – send a snapshot.
- **RequestSpotInstrumentPositions** – attribute of requesting spot positions in instruments from the QUIK server. Valid values:
  - 0 – position request is disabled;
  - 1 (by default) – position request is enabled.
- **RequestSpotCashPositions** – attribute of requesting spot positions in cash from the QUIK server. Valid values:
  - 0 – position request is disabled;
  - 1 (by default) – position request is enabled.
- **RequestFuturesClientAccountPositions** – attribute of requesting futures positions on client accounts from the QUIK server. Valid values:
  - 0 – position request is disabled;
  - 1 (by default) – position request is enabled.
- **RequestFuturesClientAccountLimits** – attribute of requesting futures limits on client accounts from the QUIK server. Valid values:
  - 0 – limits request is disabled;
  - 1 (by default) – limits request is enabled.
- **BridgeMode** – attribute of a FIX session state dependence with the QAPI connection to the QUIK server. Valid values:
  - 0 (by default) – FIX session does not depend on the QUIK server connection status;

- 1 – FIX session depends on the QUIK server connection status:
  - For the **IN**-type connection:

    If the Interface is not connected to the QUIK server when connecting the FIX client, the session is closed with the Logout message.
    If connection to the QUIK server is lost when a FIX session is active, the Interface closes it by sending a Logout. In this case the port for incoming connections is always open.
  - For the **OUT**-type connection:

    When launched, the Interface initiates a FIX session (connection and sending a Logon) only after the connection to the QUIK server is established.
    If connection to the QUIK server is lost during an active FIX session, the Interface closes it by sending a Logout. The next attempt to start a FIX session the Interface will make only after the connection to the QUIK server has been restored.

- **MLegFillReportingMode.Swap** – mode of sending reports on SWAP trades. Valid values:
  - 0 (by default) – ER message does not contain LegExecGrp group, parameters of the parent and leg trades are transmitted in the different tags on the basic level.
  - 1 – ER message contains LegExecGrp group, all parameters of the parent trades are transmitted on the basic level, parameters of the leg trades are transmitted in the group.
- **MLegFillReportingMode.RepoCpty** – mode of sending reports on REPO with CCP trades . Valid values:
  - 0 (by default) – ER message does not contain LegExecGrp group, parameters of the parent and leg trades are transmitted in the different tags on the basic level.
  - 1 – ER message contains LegExecGrp group, all parameters of the parent trades are transmitted on the basic level, parameters of the leg trades are transmitted in the group.
- **MLegFillReportingMode.FutSpread** – mode of sending report on the future spread trades. Valid values:
  - 0 (by default) – ER message does not contain LegExecGrp group, parameters of the parent and leg trades are transmitted in the different tags on the basic level.
  - 1 – ER message contains LegExecGrp group, all parameters of the parent trades are transmitted on the basic level, parameters of the leg trades are transmitted in the group.
- **MLegFillReportingMode.CcyFixing** – mode of sending report on the currency fixing trades. Valid values:
  - 0 (by default) – ER message does not contain LegExecGrp group, parameters of the parent and leg trades are transmitted in the different tags on the basic level.
  - 1 – ER message contains LegExecGrp group, all parameters of the parent trades are transmitted on the basic level, parameters of the leg trades are transmitted in the group.

2.6.10 [ClientIDValidityTokens] section

The section contains the list of validation keys and related lists of valid client codes separated by a vertical bar ('|'). This check is only run if the ClientIDValidityTokenTag setting is specified.

The format:

```
<Validation key>=<List of client codes>
…
```

For example:

```
[ClientIDValidityTokens]
12345=Q1|Q2
67890=W3
```

2.6.11 [Mail] section

This section is intended for setting up sending error messages to the system administrator via email:

- **EnableReporting** – enables the notification of administrator about errors. Valid values:
  - 0 (by default) – disabled;
  - 1 – enabled.
- **Server** – SMTP server's address,
- **Port** – SMTP server's port,
- **FromAddress** – sender's address,
- **Address** – recipient's address, Value by default: "" (empty).
- **User** – user name for SMTP authorization. Value by default: "" (empty).
- **Password** – user password for SMTP authorization. A password must not begin with "$" and "#" characters. Value by default: "" (empty). The setting value can be encrypted if necessary (for details, see 4).
- **MsgTimespan** – period of sending notifications in <HH:MM:SS-HH:MM:SS> format. Value by default: 00:00:00-23:59:59.
- **DisableRepeatingNotifications** – disable resending of notifications for recurring events. Valid values:
  - 0 (by default) – send notification about every event;
  - 1 – do not send notification if the status has not changed since the previous notification was sent.
- **ClientSchemeFileName** – path to the client XML scheme with the text messages. It is used to override the attribute values of the global XML scheme. The file format is described in 2.11. Value by default: "" (empty).

  For example:

```
[Mail]
```

```
EnableReporting=1
Server=198.125.010.020
Port=12350
FromAddress=abc@abc.ru
Address=yyy@yyy.ru
Sender=abc@abc.ru
User=user1
Password=$55555
MsgTimespan=00:00:00-23:59:59
DisableRepeatingNotifications=1
ClientSchemeFileName=Desc/UserMessages.xml
```

2.6.12 [ISIN] section

Global correspondences between ISIN codes and instruments are listed in this section. The correspondences can be redefined in [<Exchange code>] sections (for more details see 2.6.17).

The section is used for identification of instruments with the same ISIN codes in different classes (traded at different markets). If <SECBOARD> has a certain class then instrument will be exactly defined by its ISIN. The format:

```
<ISIN>=<SECCODE>,<SECBOARD>
```

For example:

```
[ISIN]
RU0008959655=TQNE,TQBR
```

2.6.13 [fix_tags] section

This section is intended for listing FIX message types which are to be ignored while logging.

For example:

```
[fix_tags]
1=0
A=0
```

2.6.14[CounterParties] section

The section contains the key which allows to identify a firm at a certain market.

```
<Market code>=<cp>
```

While generation of reports on all the orders and trades at the <Market code> market value of <cp> is put to the [76] ExecBroker tag.

For example:

```
[CounterParties]
RTS=M
```

2.6.15[TradeUserID] section

- **Enable** – key which defines the source for TradeUserID field. Valid values:
  - 0 (by default) – value of UserID from the message about trade;
  - 1 – value of UserID by default (value of Value key).
- **Value** – value of UserID by default.

  For example:

```
[TradeUserID]
Enable=1
Value=Client1
```

2.6.16[Markets] section

The section is used to set up correspondences between markets and instrument classes. Classes codes in the list are separated by a vertical bar ('|'). The format:

```
<Market code>=<Class code>
```

> **Each class can be added only to one market.**

For example:

```
[Markets]
FORTS=|SPBFUT|SPBOPT|
MICEX=|TQBR|TQOB|SMAL|
```

```
NASDAQ=|NA_ADR|NA_ETF|
NYSE=|NY_FINANCE|NY_GOODS|
```

2.6.17[<Market code>] section

Sections of [<Market code>] view, where <Market code> is one of exchanges from the [Markets] section, are used to set up correspondences between ISIN and exchange instrument codes which are specific for <Market code> market. The format:

```
<ISIN>=<SECCODE>, <SECBOARD>
```

For example:

```
[MOEX]
RU0008959655=TQBR,TQNL,TQNE,TQNB
```

2.6.18[market_<Market code>] section

The section of [market_<Market code>] view, where <Market code> is a market name, provides setting up translation of Level II Quotes table in the current market:

- **OB_EnableLevel2** – attribute of quotes transmission for <Market code> market. The option can be used for disabling Level II Quotes table in a certain market viewing for a client. Valid values:
  - 0 – quotes transmission is disabled;
  - 1 (by default) – enabled.
- **OB_MarketDepth** – quantity of the best quotes in <Market code> market. Value by default: 5. Zero value stands for displaying all the quotes.
- **LotsQty** – method of displaying security quantity. Valid values:
  - 0 (by default) – in items;
  - 1 – in lots.

> **Quantity in OMS orders is transmitted in items regardless of the value in the LotsQty parameter.**

- **MassCancelTime** – time of automatic cancellation of active orders and stop orders for classes of the <Market code> market in format <HHMMSS>. Value by default: 999999 (not defined).
- **AllowReplace** – set up allowance of order replace operations at the <Market code> market. Valid values:

- 0 – order replace is unavailable;
- 1 (by default) – available.

- **CheckSettleDate** – check for matching between settlement date specified by client in tag [64] SettlDate with settlement date of the instrument. Valid values:
  - 0 (by default) – check is disabled;
  - 1 – check is enabled.

> **The same name settings in sections [class_<Class code>] (see 2.6.19) and in section [General] (see 2.6.9) have higher and lower priority correspondently relative to the setting of this section.**

- **MergePrevAndClosePrices** – indicates that prevprice and closeprice parameters of the Quotes table are combined when transmitting the MDEntryType parameter equal to 5 in Market Data Refresh message. Valid values:
  - 0 (by default) – prevprice is only transmitted;
  - 1 – prevprice and closeprice parameters are combined when transmitting. Updating any of the parameters generates Market Data Refresh for MDEntryType=5.

  For example:

```
[market_SPBFUT]
OB_EnableLevel2=1
OB_MarketDepth=5
LotsQty=0
MassCancelTime=210000
AllowReplace=1
CheckSettleDate=0
MergePrevAndClosePrices=1
```

If the program finds a section of [market_<Market code>] view then it looks for the <Market code> market in the [Markets] section, receives list of classes (ISIN, SECCODE) at it and for each <Class code> class looks for the [class_<Class code>] section.

2.6.19 [class_<Class code>] section

The section of [class_<Class code>] view, where <Class code> is a class code, provides setting up translation of Level II Quotes table of the current class.

> **Settings in this section have higher priority than the global ones from the [General] section and those from [market_<Exchange code>].**

- **OB_EnableLevel2** – attribute of quotes transmission of <Class code> class. Valid values:

- 0 – quotes transmission is disabled;
- 1 (by default) – enabled.

- **OB_MarketDepth** – quantity of the best quotes in <Class code> class. Value by default: 5. Zero value stands for displaying all the quotes.
- **LotsQty** – method of displaying security quantity. Valid values:
  - 0 (by default) – in items;
  - 1 – in lots.

> **Quantity in OMS orders is transmitted in items regardless of the value in the LotsQty parameter.**

- **AllowReplace** – set up allowance of order replace operations with the <Class code> class. Valid values:
  - 0 – order replace is unavailable;
  - 1 (by default) – available.
- **SecurityFilter** – list of instrument codes that must be allowed or prohibited to be accessed be the Interface. Instrument codes in the list are separated by '|'. Templates '*' that allow filtering by mask are available for use. Value by default '' (empty).
  For example:

```
SecurityFilter=LKOH|MS*|*NG|A*T
```

- **SecurityFilterAction** – filtration of the instruments list in the context of class set in SecurityFilter setting. Valid values:
  - Allow – the Interface ignores all the instruments of the class except of those listed;
  - Disallow (by default) – the Interface ignores listed instruments.
- **CheckSettleDate** – check for matching between settlement date specified by client in tag [64] SettlDate with settlement date of the instrument. Valid values:
  - 0 (by default) – disable check;
  - 1 – enable check.

> **The settings with the same name in [market_<Exchange code>] (see 2.6.18) and [General] (see 2.6.9) sections have lower priority relative to the setting of this section.**

- **DefNewOrderTransInfo** – Info identifier of transaction of order entry. It is used when choosing transaction for placing order, if class has several transactions for order entry.

- **MergePrevAndClosePrices** – indicates that prevprice and closeprice parameters of the Quotes table are combined when transmitting the MDEntryType parameter equal to 5 in Market Data Refresh message. Valid values:
  - 0 (by default) – prevprice is only transmitted;
  - 1 – prevprice and closeprice parameters are combined when transmitting. Updating any of the parameters generates Market Data Refresh for MDEntryType=5.

  For example:

```
[class_ TQNE1]
OB_EnableLevel2=1
OB_MarketDepth=5
LotsQty=0
AllowReplace=1
CheckSettleDate=0
MergePrevAndClosePrices=1
```

2.6.20 [CodeToAccount] section

[CodeToAccount] section is intended for setting up conversion of values from orders to values which are sent to the trading system accordingly with the CodeToAccount option from the [General] section (see 2.6.9).

Correspondences are to be set up in the following way: <AAA>=<BBB>, where <AAA> – value from order, <BBB> – value which is sent to the trading system.

For example:

```
[CodeToAccount]
Client1=389_000
Account1=389_001
```

Example of section's setting if the value of parameter CodeToAccount in section [General] (see 2.6.9) is 6:

```
[CodeToAccount]
L01-00000F00=NC0038900000
```

Where:

- L01-00000F00 – trading account;
- NC0038900000 – partner code in QUIK.

2.6.21 [CurrCodes] section

The section is used to set up the correspondence between currency codes which are used in the system (<Currency code>) and international currency codes (<Currency ISO code>). The format:

```
<Currency code>=<Currency ISO code>
```

For example:

```
[CurrCodes]
SUR=RUB
```

2.6.22 [ShortNames] section

The section is used to set up correspondences between instrument codes and alternative short names. The format:

```
<Instrument code>=<Short name>
```

For example:

```
[ShortNames]
RU14TATN3006=TATN
```

2.6.23 [ContraBrokers_<Market code>] section

Sections of [ContraBrokers_<Market code>] view, where <Market code> is one of the markets from the [Markets] section, contain the correspondences between firm names at the QUIK server and firm names which are used by users of Interface when they send address orders and in execution reports on address orders. Lists for replacement are separate for each market. The format:

```
<user firm>=<firm on the QUIK server>
```

For example:

```
[ContraBrokers_<MFD>]
130150529 =NC0038900000
```

2.6.24 [MemoOrderId] section

This section allows using the comment to identify if the order/trade refers to the selected Interface. OrderIdShift option allows specifying the start shift when using several Interfaces with the same order identifier in one system. The principle of the option is in order identifiers shift so that they do not coincide in different adapters. The functional of marking orders works only under the condition that the CommentType setting in the [General] section is equal to 2 (see 2.6.9).

- **QuikFixId** – symbolic identifier which is unique for each Interface connected to the server. By default, the parameter is empty.
- **Classes** – list of classes separated by '|' which have identifier in the comment to order. By default, the parameter is empty.
- **OrderIdShift** – shift of order identifiers.
- **OrderIdSize** – capacity of order identifier. Valid values are integer numbers in the [3; 12] interval. Value by default: 3.
  For example:

```
[MemoOrderId]
QuikFixId=E
Classes=TQNE|TQBR
OrderIdShift=10
OrderIdSize=6
```

2.6.25 [ISINConvert] section

The section is used for replacement of instrument ISIN codes in execution reports if instrument identification by ISIN code is set up.

```
<ISIN code>=<New ISIN code>
```

For example:

```
[ISINConvert]
RU0008959655= RU0008959650
```

2.6.26 [OMSClasses] section

The [OMSClasses] section contains the list of supported OMS classes and corresponding XML files. Format of section:

```
<OMS class code at the QUIK server>=<XML file name>
```

For example:

```
[OMSClasses]
OMS=Desc/oms.xml
```

2.6.27 [OMSExchanges] section

The section is intended for correct instrument defining, when OMS order is registered in the configuration with an OMS class containing few instruments with the same ISIN/ticker, and different markets/currencies. In this section correspondences between exchange codes from the tag [100] ExDestination of incoming message and exchange codes in the OMS Manager Module are specified.

When a new OMS order is registered, the instrument is searched by its ISIN and additionally by the exchange code (tag [100] ExDestination), ticker (tag [55] Symbol) and currency (tag [15] Currency). If the instrument is not found by any of these parameters, or few instruments found, then the OMS order is created in Bad status.

> **This searching algorithm is applied only in cases, when a FIX client identifies an instrument by ISIN (SecurityIDSource22=4).**

Correspondences are specified in the following format:

```
<Exchange code in OMS Manager Module> = <ExDestination1> | <ExDestination2> |… |
<ExDestinationN|
```

For example:

```
[OMSExchanges]
L=L|XLON
US=N|O|XNAS|XNYS
```

2.6.28 [OMS] section

- **SettleDateReportingMode** – transmission mode for the settlement date by OMS orders in the ExecutionReport messages. Valid values:
  - 0 (by default) – trade settlement date is transmitted in the ExecutionReport messages on trades; it is not filled in other messages;
  - 1 – trade settlement date is transmitted in the ExecutionReport messages on trades; OMS order settlement date is transmitted in other messages.
- **CommissionReporting** – defines data transmission for an OMS order in the ExecutionReport messages. Valid values:

- 0 (by default) – type and amount of the commission are not transmitted;
- 1 – commission type is transmitted in the tag [13] CommType, the amount is transmitted in the tag [12] Commission.

For example:

```
CommissionReporting=1
```

- **HistoricalOrderThreshold** – number of days from the day of the registration day, during which the OMS order data is saved by the Interface. Value by default: 365.
- **AvgPricePrecisionMode** – accuracy of the transmitted weighted average price. Valid values:
  - 0 (by default) – value is determined as <OMS instrument accuracy> + 2 (but not less than 6 digits;
  - 1 – OMS instrument accuracy.

2.6.29 [ExecReportOrdTypes] section

The section is intended for setting up substitutions of values of tag [40] OrdType for values selected by user. These substitutions are applied for sending ExecutionReport messages.
The section format:

```
<Value of OrdType tag>=<Value which is used when message is sent>
```

For example:

```
[ExecReportOrdTypes]
R=2
U=2
```

2.6.30 [RIC] section

This section is intended for setting correspondences between RIC and ISIN codes of instruments in format:

```
<RIC Code>=<ISIN code>
```

For example:

```
[RIC]
RIC_MKOH=RU0000000000
RIC_LKOH=RU0000000001
RIC_MSNG=RU0000000002
```

2.6.31 [ClientsForConvert] section*

The section contains the list with codes of clients whose trades are needed to be converted, in format:

```
[ClientsForConvert]
ClientCode1
...
ClientCode2
```

2.6.32 [ClassesForConvert] section*

The section contains the list of classes whose trades are needed to be converted, in format:

```
[ClassesForConvert]
<ClassCode_1>
...
<ClassCode_N>
```

2.6.33 [CurrClasses_<Currency>] section*

The section contains the parameters of the Fx conversion on currency <Currency> in format:

```
Default=<fx_classcode>,<fx_seccode>
ClientCode_1=<fx_classcode>,<fx_seccode>
...
ClientCode_N=<fx_classcode>,<fx_seccode>
```

Where:

- **ClientCode** – client code in the client trade;
- **fx_classcode** – class code of Fx instrument;
- **fx_seccode** – code of Fx instrument;

2.6.34 [ConvSettings] section*

The section contains the fixed parameters of Conv orders.

- **Class** – code of a class in which Conv trades are executed.

  For example:

```
Class=BQUOTE4
```

> **(*) The [ClientsForConvert], [ClassesForConvert], [ConvSettings] and [CurrClasses_<Currency>] sections are to be set up if the trades are reported via the Interface which passed the currency conversion with the help of FX Convertor module. The values of the options must coincide with the same options in FxConverter.ini of FX Convertor module.**

2.6.35 [TimeDiffThresholds] section

The section contains threshold values of difference between the current time and time parameters of incoming FIX messages. When threshold values are exceeded the incoming message is rejected by the Interface with sending a Reject message.

- **SendingTime** – threshold value (in seconds) of difference between current local time in GMT and value of tag [52] SendingTime of any incoming message. Value by default: '0' (checking is disabled).
- **OrigSendingTime** – threshold value (in seconds) of difference between current local time in GMT and value of tag [122] OrigSendingTime of incoming message with attribute [43] PossDupFlag=Y. Value by default: 0 (checking is disabled).
- **TransactTime** – threshold value (in seconds) of difference between current local time in GMT and value of tag [60] TransactTime of incoming messages NewOrderSingle and OrderCancelReplaceRequest. Value by default: 0 (checking is disabled).
  Example:

```
SendingTime=0
OrigSendingTime=2
TransactTime=0
```

2.6.36 [FXWorkflow] section

> **For information about specifics of external systems, contact QUIK technical support: quiksupport@arqatech.com.**

The section contains parameters used to support FX operations (quotes request by volume, FX orders entry, execution reporting).

- **ClientFirmSourceTag** – number of the tag the value of which is interpreted as client's firm code at the broker. If the value is specified, then the Interface uses makler's quote request and order entry transactions while transferring the tag value as a firm. Value by default: 0.

- **RestrictQuoteUPS** – a restriction on the number of quote updates per second. Value by default: 0 (number of updates is not limited). Valid values: from 1 to 100.

2.6.37 [CentPresentation] section

The section contains correspondences between codes of basic currencies and accuracy codes of monetary unit in format:

```
<Basic currency code>=<Accuracy code of monetary unit>
```

Example:

```
[CentPresentation]
USD=CNT
GBP=GBX
```

2.6.38 [FIDNClasses] section

The section contains the list of address classes, at which multi-day orders can be placed.

Format:

```
[FIDNClasses]
<CLASSCODE1>
<CLASSCODE2>
…
<CLASSCODEN>
```

Example:

```
[FIDNClasses]
PSAU
```

2.7 quikapi.ini file format

2.7.1 [connect] section

- **trade-server-ip-address** – IP address of QUIK server.
- **port** – port of QUIK server.
- **auth-name** – key of QUIK server.

- **check-tmo** – time interval (in seconds), after which QUIK server will break the connection to Interface if some trouble with connection appears.

2.7.2 [transport] section

- **module** – value of the key is 'wt_dtcp.dll'.

2.7.3 [proxy] section

- **use-proxy** – attribute of proxy server usage. Valid values:
  - 0 (by default) – usage of proxy server is prohibited;
  - 1 – usage of proxy server is allowed.
- **proxy-server-ip-address** – IP-address of proxy server.
- **proxy-port** – port of proxy server.
- **proxy-protocol** – protocol which is used for connection. Valid values:
  - 'SSL without authentication' or '1';
  - 'SOCK5 without authentication' or '2' (by default);
  - 'SOCK5 with login/password' or '3';
  - 'SSL with login/password (basic access authentication)' or '4'.
- **proxy-user-name** – user name for authentication at proxy server.
- **proxy-user-password** – password for authentication at proxy server.
- **get-password-mode** – mode of user name and password reading for authentication at proxy server. Valid values:
  - 0 – user name and password are entered manually;
  - 1 (by default) – values are taken from proxy-user-name and proxy-user-password parameters.

  Example of QuikAPI.ini file:

```
[connect]
trade-server-ip-address=192.168.2.254
port=7894
auth-name=QUIK_BROKER
check-tmo=150

[transport]
module=wt_dtcp.dll

[proxy]
use-proxy=1
proxy-server-ip-address=192.168.1.1
proxy-port=1234
proxy-protocol=1
proxy-user-name=user123
```

```
proxy-user-password=qwert
get-password-mode=0
```

2.8 Configuration of digital signature

The following table contains a list of supported cryptographic providers, their minimum versions and required external software for digital signature:

QUIK cryptographic provider	Minimum version	External software for digital signature
SignalCom_Pr	11.1.4	MessagePro 3.2.0+
QUIK MP_Pr	2.6.1	CryptoPro CSP 3.9.8227+

To set up appliance of digital signature in the Interface, copy the following files to the FIX catalog:

- signalcom.dll;
- mespro.dll;
- signalcom.ini.

Example of digital signature setting for the SignalCom cryptographic provider

1. Configure settings of digital signature in [General] section of quikfix.ini file:
   - EnableDigitSign – attribute of using the digital signature. Valid values:
     - 0 (by default) – usage of digital signature is disabled;
     - 1 – usage of digital signature is enabled.
   - DigitSignIni – name of configuration file of digital signature. Value by default is an empty string.

   For example:

```
[General]
EnableDigitSign=1
DigitSignIni=SignalCom.ini
```

2. Configure settings of digital signature in quikapi.ini file:

```
[CryptoProviders]
SignalCom=1

[CryptoProvidersSettings]
SignCryptoProvider=SignalCom
```

```
[SignalCom]
Module=./signalcom_pr.dll
IniFile=./signalcom.ini
```

3. In signalcom.ini cryptoprovider's configuration file set the following values of the keys:

   _ in [signal_com_message_pro_common_settings] section:

```
[signal_com_message_pro_common_settings]
certificates_path=./PKI/certs
CRLs_path=./PKI/crl
CAs_path=./PKI/CA
input_data_format=FORMAT_PEM
output_data_format=FORMAT_PEM
```

   _ in [signal_com_message_pro_sign_settings] section:

```
[signal_com_message_pro_sign_settings]
pse_path=./PKI/
sign_cert_file_name=./PKI/CERT/tcert_9140.cer
sign_key_file_name=
```

   _ where PKI – directories with certificates.

2.9 Configuration of FIX messages

XML-scheme is intended to easily configure the format of messages sent and received by the Interface. Each FIX connection can use its own scheme. The scheme does not allow adding new tags to the translation, which are not described in the protocol, but allows:

- prohibiting translation of the tag, described in the protocol;
- copying the value of the tag to another (which is not included in the protocol);
- moving the value of the tag to another (which is not included in the protocol);
- setting up the list of substitutions for the value of the tag to be transmitted or received;
- specifying the condition when an incoming message is declined;
- transmitting unusual tags (which are absent in the protocol) in answering Execution Report messages;
- using regular expressions in commands (according to common ECMAScript syntax).

2.9.1 Format of scheme file

```
    <Rule MessType="{FIX message type}">
```

```
        <Tag ID="{tag number}" AllowedNonStandardValues="{allowed values of
enumerated tags}">
    <Email/>
    <Remove/>
    <Reject If="{tag value}" Rx="{regular expression}" CompareOperation="{comparison
operation type}" Message="{the reason of reject}"/>
    <Replace If="{tag value}" Rx="{regular expression}" Then="{substitution}"
CompareOperation="{comparison operation type}"/>
    <CustomReplace If="{predetermined condition}" Then="{substitution}"/>
    <ExternalReplace RefTagID=”{tag number for condition}”>
       <ExecTrigger If="{Tag ID tag value}" Rx="{regular expression}"/>
       <Condition If=”{obtained value in RefTagID}” Rx="{regular expression}"
Then=”{substitution}”
       CompareOperation="{comparison operation type}"/>
    </ExternalReplace>…
    <MultiExternalReplace>
      <CompositeCondition Then="{substitution}" LogicalOperation="{AND|OR}">
         <Condition RefTagID="{the external tag number of condition}" If="{the
RefTagID value}" Rx="{Regular expression}"/>
      </CompositeCondition>
    </MultiExternalReplace>
    <Swap CounterTag=”{target tag value}”>
            <Condition If="{Value received in TagID or RefTagID}" Rx="{regular
expression}"
       CompareOperation="{comparison operation type}" RefTagID="{number of external
       tag for the condition}" />
          </Swap>
    <Clone Target="{target tag number}" IfExistsAction="{action}"
    Direction="{message direction}" DropSource="{Y/N}">
      <Condition If="{Value received in TagID or RefTagID}" Rx="{regular expression}"
       CompareOperation="{comparison operation type}" RefTagID="{number of external
tag for the condition}" />
    </Clone>
   <Set Value="{substitution}"/>
  </Tag>
</Rule>
```

Where:

- {FIX message type} – type of FIX message (HeartBeat=0, Test=1...). Number of tags, which are always included in each header message, are separated into a single block. MessType = ‘!’ is used for it.
- {comparison operation type} – type of comparing operation. Valid values:
  - IsEqual (by default) – ‘equal’ operation is used for comparing;

- IsNotEqual – 'not equal' operation is used for comparing.

- {the reason of reject} – text that is sent to the FIX client in the [58] tag when the message is rejected according to the conditions of the <Reject> element.
- <Rule> – element is used to set various commands.

<Rule> element commands and attributes

- **Remove** – command is used to exclude the tag from a message type. For example:

```
<Rule MessType="8">
	<Tag ID="47">
		<Remove/>
	</Tag>
</Rule>
```

- **Email** – command is used to make the tag included to the text of email alert use the <Email> command. For example:

```
<Rule MessType="8">
	<Tag ID="1">
		<Email/>
	</Tag>
	<Tag ID="11">
		<Email/>
	</Tag>
</Rule>
```

- **Replace** – command is used to set the list of substitutions for the tags to be transmitted. The values of 'if' attribute must be unique within a tag. If several same values are included in 'if' attribute then all the substitutions except for the first one in the row are ignored. For example:

```
<Rule MessType="8">
	<Tag ID="63">
		<Replace If="T0" Then="1" CompareOperation="IsEqual"/>
		<Replace If="T02" Then="3" CompareOperation="IsEqual"/>
		<Replace If="B02" Then="3" CompareOperation="IsEqual"/>
		<Replace If="T03" Then="4" CompareOperation="IsEqual"/>
		<Replace If="B03" Then="4" CompareOperation="IsEqual"/>
	</Tag>
</Rule>
```

- **CustomReplace** – command is used to substitute values of tags containing Cyrillic characters by a fixed value. For example:

```
<Rule MessType="8">
      <Tag ID="58">
            <CustomReplace If="ContainsCyrillic" Then="Transaction checking
      failed"/>
      </Tag>
</Rule>
```

- **ExternalReplace** – command is used to replace a tag by condition.
  Examples:
  - Replacement of a tag when a condition set in ExecTrigger attribute is satisfied. If the condition is satisfied, the tag is replaced with the tag specified in ExternalReplace command:

```
  <Tag ID="120">
  <Replace If="GBp" Then="GBX"/>
  <ExternalReplace RefTagID="15">
  <ExecTrigger If=""/>
  <Condition If="GBX" Then="GBP"/>
  </ExternalReplace>
  </Tag>
```

  - Substitution of the tag value by condition for the value of another tag.
  - The example in which the value of the [59] tag is equal to 4 if the value of the [18] tag is G:

```
<Rule MessType="D">
  <Tag ID="59">
        <ExternalReplace RefTagID="18">
              <Condition If="G" Then="4" CompareOperation="IsEqual"/>
        </ExternalReplace>
  </Tag>
</Rule>
```

- **MultiExternalReplace** – command is used to substitute the tag value by condition for the values of other tags. The following attribute is used:
  - **LogicalOperation** – logical operation that is used to calculate the truth of the compound condition the attribute is provided. Possible values:
    - AND – compound condition is considered fulfilled when all the nested conditions are fulfilled;

- OR – compound condition is considered fulfilled when at least one nested condition is fulfilled.

- The example in which the value of the tag [30] ExecMarket is replaced only for ER by executions for certain clients (compound condition on tag [109] ClientID and tag [150] ExecType):

```
<Rule MessType="8">
      <Tag ID="30">
        <MultiExternalReplace>
          <CompositeCondition Then="MARKET1" LogicalOperation="AND">
            <Condition RefTagID="109" If="CL01"/>
            <Condition RefTagID="150" Rx="1|2"/>
          </CompositeCondition>
          <CompositeCondition Then="MARKET2" LogicalOperation="AND">
            <Condition RefTagID="109" If="CL02"/>
            <Condition RefTagID="150" Rx="1|2"/>
          </CompositeCondition>
        </MultiExternalReplace>
      </Tag>
</Rule>
```

- **Swap** – command is used to switch the values of current and specified tags.
  The example in which the values of [31] and [5007] tags are switched if the value of [1] tag is equal to RCB:

```
<Rule MessType="8">
      <Tag ID="31">
             <Swap CounterTag="5007">
                    <Condition RefTagID="1" If="RCB"/>
             </Swap>
      </Tag>
</Rule>
```

Only one <Swap> command can be specified for one tag.

1. **If one of tags to be switched is missing, the operation is carried out anyway. The value of the tag which is present is moved to a new tag, then thesource (present) tag is deleted.**
2. **When the list of conditions is specified by the values of current or external tags, the values are switched only if any of the conditions is true.**
3. **The tag values are switched using the <Swap> command at the last step, but before processing the <DiscardWholeMessage> command.**

- **Reject** – command is used to reject messages by the specified condition.

  The command works for four types of incoming messages:

  - New Order Single;
  - Order Cancel Replace Request;
  - Order Cancel Request;
  - Quote Request.

  The example in which a new order query is rejected if the value of the tag [64] SettlDate is not equal 20130401:

```
<Rule MessType="D">
      <Tag ID="64">
            <Reject If="20130401" CompareOperation="IsNotEqual" Message="Incorrect
value of settlement date"/>
      </Tag>
</Rule>
```

- **DiscardWholeMessage** – command is used to block message sending by a specified condition.
  The example in which messages are set blocked if the condition is true:

```
<Rule MessType="8">
      <Tag ID="39">
            <DiscardWholeMessage If="A" CompareOperation="IsEqual"/>
      </Tag>
</Rule>
```

- **Clone** – command is used to copy a tag value to another tag. The following attributes are used:

  - **DropSource** is used to manage the source tag. Possible values:
    - Y – delete the source tag from a message;
    - N – keep the source tag in a message.

  - For example, if the value, which is transmitted in tag [100], is necessary to be additionally transmitted in tag [1100] and the [100] source tag is to be kept in a message, the section must be configured in the following way:

```
<Rule MessType="8">
  <Tag ID="100">
        <Clone Target="1100" DropSource="N"/>
  </Tag>
</Rule>
```

- If the target tag ([1100]) already exists in the <Rule> element then its value is not to be rewritten and a double tag is added to the message. To change the behavior, use the IfExistsAction attribute.
- **IfExistsAction** – behavior in case of a double tag. Possible values:
  - Add (by default) – add the target tag to the message even if there is the same one in the message;
  - Skip – do not add a tag if there is the same one in the message;
  - Replace – replace the existing tag.

  For example:
  - Moving the value from tag [58] to [1158] without duplicating:

```
<Rule MessType="8">
	<Tag ID="58">
		<Clone Target="1158" DropSource="Y" IfExistsAction="Replace"/>
	</Tag>
</Rule>
```

  - Moving the value from tag [15] to [1158]. If there are no tag [15], then moving the value from tag [58] to [1158]:

```
<Rule MessType="8">
	<Tag ID="15">
		<Clone Target="1158" DropSource="Y" IfExistsAction="Skip"/>
	</Tag>
	<Tag ID="58">
		<Clone Target="1158" DropSource="Y" IfExistsAction="Skip"/>
	</Tag>
</Rule>
```

- **Direction** – direction to which the substitution and copying are applicable. Possible values:
  - Out (by default) – copying/substitution are performed only for outgoing messages;
  - In – copying/substitution are performed only for incoming messages;
  - InOut – copying/substitution are performed for both incoming and outgoing messages.

  For example:

```
<Rule MessType="D">
	<Tag ID="60">
```

```
            <Clone Target="109" DropSource="Y" IfExistsAction="Replace"
            Direction="In" />
      </Tag>
</Rule>
<Rule MessType="8">
      <Tag ID="109">
            <Clone Target="60" DropSource="Y" IfExistsAction="Replace"
                  Direction="Out" />
      </Tag>
</Rule>
```

> **The Target values in <Clone> command in a single message cannot be repeated. If any recurrences exist for Clone.Target, all the commands except the first one in the description are ignored.**

Move of tag values on a specified condition. The example in which the values of the [12] and [13] tags are moved to the [5061] and [5062] tags correspondingly, if the [13] tag value is 1:

```
<Rule MessType="D">
      <Tag ID="12">
            <Clone Target="5061" DropSource="Y" IfExistsAction="Replace">
                  <Condition If="1" RefTagID="13" />
            </Clone>
      </Tag>
      <Tag ID="13">
            <Clone Target="5062" DropSource="Y" IfExistsAction="Replace">
                  <Condition If="1" />
            </Clone>
      </Tag>
</Rule>
```

- **Echo** – command is used to transmit unusual tags (which are absent in the protocol) in answering Execution Report and Quote messages.
  The example in which the tag [25000] is received in all Execution Report messages by the new order:

```
<Rule MessType="D">
      <Tag ID="25000">
            <Echo/>
      </Tag>
</Rule>
```

- **AllowedNonStandardValues** – attribute is used to set unusual values of enumerated tags that are to be transmitted.
  The example in which the tag [378] that is supported starting with FIX 4.3 is made available for transmission in FIX 4.2:

```
<Rule MessType="8">
      <Tag ID="378" AllowedNonStandardValues="6" />
</Rule>
```

  Configuration of unusual tags (which are absent in the protocol) transmission.
  The example in which tag [109] is made available for transmission:

```
<Rule MessType="9">
      <Tag ID="109" />
</Rule>
```

- **Set** – command is used to unconditionally set a constant tag value.
  The example in which the unconditional type of instrument identification by ISIN is set in the New Order Single messages:

```
<Rule MessType="D">
      <Tag ID="22">
            <Set Value="4"/>
      </Tag>
</Rule>
```

- **ForwardToTransField** – command is used to configure transmission of a tag to a transaction field being addressed by an Info identifier.
  The example in which the tag [6042] is set to be transmitted to the transaction field with the identifier 5005:

```
<Rule MessType="D">
  <Tag ID="6042">
    <ForwardToTransField FieldInfo="5005"/>
    …
  </Tag>
</Rule>
```

Usage of regular expressions in commands

To use regular expressions in conditional commands (according to common ECMAScript syntax) the **Rx** attribute can be specified for the following elements supporting the **If** attribute:

- Reject If;
- Replace If;
- ExternalReplace ExecTrigger If;
- ExternalReplace Condition If;
- Swap Condition If;
- Clone Condition If.

An example in which the [55] Symbol tag is replaced by a predefined value if the value specified by a client meets the following conditions:

- consists of five characters and starts with T;
- the second and third characters are arbitrary;
- the fourth character is the number in the range: 0, 1 or 2;
- the fifth character is any number.

```
<Rule MessType="D">
  <Tag ID="55">
    <Replace Rx="^T.{2}[0-2]{1}\d{1}$" Then="TATN" />
  </Tag>
</Rule>
```

> **If a message contains Rx and If attributes simultaneously, then the Rx attribute has higher priority.**

Usage of commands depending on message direction

- Y – command is used;
- N – command is not used.

Command	IN	OUT	Description
Email	N	Y	Addition of a tag containing a value to the text of email messages notifying of errors occurred while entering and cancelling orders. Only for ExecutionReport, CancelReject messages
Remove	N	Y	Tag transmission is prohibited
Reject	Y	N	Rejection of an incoming message if the condition is met. Only for following messages: _ New Order Single; _ Order Cancel Replace Request; _ Order Cancel Request;

Command	IN	OUT	Description
			_ Quote Request
DiscardWholeMessage	N	Y	Do not send a message if the condition is true
Replace	Y	Y	Replacement of the tag value if the condition is met
CustomReplace	Y	Y	
ExternalReplace	Y	Y	Replacement of the tag value if the condition is met for another tag
MultiExternalReplace	Y	Y	Replacing the tag value on a set of conditions for two or more external tags
Clone	Y	Y	Copying the tag value to another tag
Swap	Y	Y	Switching the values of the current and specified tags
Echo	Y	Y	Transmit unusual tags (which are absent in the protocol) in answering Execution Report and Quote messages. Only for NewOrderSingle, OrderCancelReplaceRequest, QuoteRequest messages
Set	Y	Y	Unconditional setting a constant tag value
ForwardToTransField	Y	N	Transmission of a tag value to a transaction field being addressed by an Info identifier. Only for NewOrderSingle, OrderCancelReplaceRequest and QuoteRequest messages

> **If the sequence of the Replace, CustomReplace and ExternalReplace commands exists, the following rules are applicable:**
>
> 1. **The replacement conditions are checked in the order the commands are listed in the scheme.**
> 2. **If the first condition is met and the replacement is made, other conditions will not be checked.**

2.9.2 Search template

The XML scheme provides the search template '*' that allows using comparison by mask for the following attributes of the scheme:

- Replace If;
- ExternalReplace Condition If;
- Reject If.

Template can be set in the beginning, middle (repeatedly) or in the end of the value. It allows setting any filters (prefix, suffix, common parts).

Example:

```
<Rule MessType="8">
            <Tag ID="58">
                  <Replace If="*(133)*" Then="(133) Security is not currently
trading"                  CompareOperation="IsEqual"/>
                  <Replace If="*(134)*" Then="(134) Trading in security is
finished"                  CompareOperation="IsEqual"/>
            </Tag>
</Rule>
```

2.9.3 Error messages

When launching the FIX adaptor, the file format of the scheme is checked for the correctness. If the file is not found or the file format is incorrect then the FIX adaptor exits and the following diagnostic messages are written to the log file:

- The file is not found:

```
[ERROR] FIX-connection rules file does not exist <path='<path>'>
```

- The file format is incorrect:

```
[ERROR] Couldn't load schema from file '<path>' [Reason='<reason, error
description>']
```

- The same target tag of the <Clone> command is set for more than one tag in a single message:

```
[Warning] Clone target duplicate was detected for message '<message type>'. Clone
command for tag #<tag number> has been left, duplicate for #<tag number> has been
omitted.
```

2.10 SecInfo.xml file format

SecInfo.xml file is intended for setting correspondences between instruments and their alternative codes.

Correspondences are to be set up in the **Securities** node with the help of the following attributes:

> **The following attributes must be set for instrument identification: either ISIN or ClassCode + SecCode.**

Attribute name	Type	Value
ClassCode	string	Code of instrument class
SecCode	string	Code of instrument
ISIN	string	ISIN code of instrument
ISOCode	string	ISO code of instrument
Tenor	string	Tenor
SEDOL	string	SEDOL code of instrument
RIC	string	RIC code of instrument
CUSIP	string	CUSIP code of instrument

For example:

```
<Securities>
      <Security ClassCode="CETS" SecCode="USD000UTSTOM" ISOCode="USD/RUB"/>
      <Security ClassCode="CETS" SecCode="EUR_RUB_TOD" ISOCode="EUR/RUB"
Tenor="TOD"/>
      <Security ClassCode="CETS" SecCode="EUR_RUB_TOM" ISOCode="EUR/RUB"
Tenor="TOD"/>
      <Security ClassCode="OMS" SecCode="LKOH" SEDOL="0040868"/>
      <Security ClassCode="NYSE" SecCode="MSFT" CUSIP="594918104"/>
      <Security ISIN="GB00B0SWJX34" SEDOL="B0SWJX3"/>
</Securities>
```

2.11 ClientEmailMessages.xml file format

ClientEmailMessages.xml file is intended for changing the predefined settings of the XML scheme of email notifications. The path to the file is defined by the ClientSchemeFileName setting in the [Mail] section. This scheme allows solving the following tasks:

- Standardization of settings for email notifications;
- Configuration of personal message theme for different events;
- Configuration of lists of recipients for different events;
- Correction of texts of messages;
- Switching off sending of individual notifications.

2.11.1 Format of scheme file

Attributes of a distribution group:

- GroupName – name of distribution group;
- Recipient – list of message recipients of distribution group separated by comma;
- Subject – message name.

Attributes of message:

- Code – number of message;
- MsgName – short name of message;
- Severity – parameter used to filter emails;
- Recipient – list of recipients separated by comma;
- Subject – message name;
- CDATA – tag, which is located inside the text message.

Template parameters can be specified in the message. When sending a message, the template parameters are replaced with the appropriate lines.

Matching templates and strings:

- {TIME} – local time;
- {DISCONNECTREASON} – cause of the switching off/brake of session;
- {PENDINGTRANS} – list of unfinished identifiers at the time of switching off transactions;
- {CLORDID} – client identifier of the request to submit, to cancel, to amend order;
- {TRANSTYPE} – transaction type on client request;
- {FIXSESSIONKEY} – identifier of FIX session in SenderCompID_TargetCompID format;
- {IDSOURCE} – parameter of identification of instrument, pointed by FIX client;
- {SECURITYID} – parameter of identification of instrument, pointed by FIX client;
- {SYMBOL} – parameter of identification of instrument, pointed by FIX client;
- {EXDESTINATION} – parameter of identification of instrument, pointed by FIX client;
- {EMAILTAGS} – list of tags with values that is included in notifications of rejection of the client order;
- {REJECTREASON} – reason for rejection of the client order;
- {TAGID} – name or number of tag of outgoing FIX message;
- {TAGVALUE} – tag value of outgoing FIX message;
- {TIMEOFPREVIOUSFIXSESSIONEND} – local time of the closure of the preceding FIX session;
- {TIMEOFSESSIONCHANGETEST} – local time of the session change check (current launch);
- {CONFIGUREDCLEARINGTIME} – configured time of the session change.

2.11.2 Supported table identifiers and their templates

- **PendingTransList** – table with the list of identifiers of transactions {CLORDID}, {TRANSTYPE} not completed at the moment of disconnection.
- **EmailTagsList** – table with a list of tags with values included in notifications about client request {TAGID}, {TAGVALUE} rejection.

Template example:

```
<Message Code="1" MsgName="FIXSessionEstablished" Severity="High" Subject="FIX
connection status notification ">
<![CDATA[{TIME}.
FIX session '{FIXSESSIONKEY}' is established.]]>
</Message>
<Message MsgName="FIXLogonFailed" Severity="High" Subject="FIX Logon failed">
      <![CDATA[{TIME}. {DISCONNECTREASON}.]]>
</Message>
```

Example of individual configuration of notifications:

```
<Message MsgName="FIXSessionEstablished" Subject="New FIX session started">
<![CDATA[{TIME}.
FIX-session '{FIXSESSIONKEY}' is started.]]>
</Message>
<Message MsgName="FIXLogonFailed" Severity="Off"/>
```

2.11.3 Integrated parameter templates

```
<?xml version="1.0" encoding="windows-1251"?>
<Messages Version="3.34.0.0">
<Group GroupName="QuikConnectivity">

<Message MsgName="ConnectedToQuikServer" Severity="High" Subject="FIX2QUIK connection
 status notification">
  <![CDATA[{TIME}.
  Connection to QUIK server is established.]]>
 </Message>

<Message MsgName="DisconnectedFromQuikServer" Severity="High" Subject="FIX2QUIK conne
ction status notification">
  <![CDATA[{TIME}.
  Connection to QUIK server lost. Description: {DISCONNECTREASON}.]]>
 </Message>

<Message MsgName="DisconnectedFromQuikServerWithPendingTrans" Severity="High" Subject
="FIX2QUIK connection status notification">
  <![CDATA[{TIME}.
  Connection to QUIK server lost. Description: {DISCONNECTREASON}.
```

```
Please note that the following transactions were in an intermediate state at the time
 of communication failure:
  {PENDINGTRANS}.]]>
 </Message>

<Message MsgName="CouldNotConnectToQuikServer" Severity="High" Subject="FIX2QUIK conn
ection status notification">
  <![CDATA[{TIME}.
  Can't connect to QUIK server. Error description: {DISCONNECTREASON}.]]>
 </Message>
</Group>

<Group GroupName="FIXConnectivity">

<Message MsgName="FIXOutSessionDisconnected" Severity="High" Subject="FIX connection
status notification">
  <![CDATA[{TIME}.
  FIX out-session has been disconnected and destroyed.]]>
 </Message>

<Message MsgName="FIXInSessionDisconnected" Severity="High" Subject="FIX connection s
tatus notification">
  <![CDATA[{TIME}.
  FIX in-session has been disconnected and destroyed.]]>
 </Message>

<Message MsgName="FIXSessionEstablished" Severity="High" Subject="FIX connection stat
us notification">
  <![CDATA[{TIME}.
  FIX session '{FIXSESSIONKEY}' is established.]]>
 </Message>

<Message MsgName="FIXSessionDropped" Severity="High" Subject="FIX connection status n
otification">
  <![CDATA[{TIME}.
  FIX session '{FIXSESSIONKEY}' is dropped by adapter.
  Reason: {DISCONNECTREASON}.]]>
</Message>
 <Message MsgName="FIXLogonFailed" Severity="High" Subject="FIX Logon failed">
  <![CDATA[{TIME}.
  {DISCONNECTREASON}.]]>
 </Message>
</Group>

<Group GroupName="ClientRequestRejects">
```

```
<Message MsgName="InvalidSecuritySpecification" Severity="High" Subject="Invalid secu
rity specification">
  <![CDATA[{TIME}.
  Specified security is not found:
  IDSource: '{IDSOURCE}'.
  SecurityID: '{SECURITYID}'.
  Symbol: '{SYMBOL}'.
  ExDestination: '{EXDESTINATION}'.]]>
 </Message>

<Message MsgName="AmbiguouslySecuritySpecification" Severity="High" Subject="Invalid
security specification">
  <![CDATA[{TIME}.
  Ambiguously security specification:
  IDSource: '{IDSOURCE}'.
  SecurityID: '{SECURITYID}'.
  Symbol: '{SYMBOL}'.
  ExDestination: '{EXDESTINATION}'.]]>
 </Message>

<Message MsgName="NewOrderRejected" Severity="High" Subject="NewOrderRequest rejected
">
  <![CDATA[{TIME}.
  Reject reason: {REJECTREASON}.
  Request details:
  {EMAILTAGS}.]]>
 </Message>

<Message MsgName="ReplaceOrderRejected" Severity="High" Subject="ReplaceOrderRequest
rejected">
  <![CDATA[{TIME}.
  Reject reason: {REJECTREASON}.
  Request details:
  {EMAILTAGS}.]]>
 </Message>

<Message MsgName="CancelOrderRejected" Severity="High" Subject="CancelOrderRequest re
jected">
  <![CDATA[{TIME}.
  Reject reason: {REJECTREASON}.
  Request details:
  {EMAILTAGS}.]]>
 </Message>
```

```
<Message MsgName="MarketDataRequestRejected" Severity="High" Subject="MarketDataReque
st rejected">
  <![CDATA[{TIME}.
  Reject reason: {REJECTREASON}.
  Request details:
  [table.id(EmailTagsList).
column(Tag, {TAGID}).
column(Value, {TAGVALUE})]
]]>
 </Message>
</Group>
<Group GroupName="Configuration">
 <Message MsgName="PossiblyIncorrectClearingTimeDetected" Severity="High"
Subject="Configuration notification">
  <![CDATA[{TIME}.
  Possibly incorrect session change time setting value has been diagnosed.
  Consider check the correctness of the value of setting 'General.ClearingTime'.
  Configured clearing time: {CONFIGUREDCLEARINGTIME}
  Time of previous FIX-session end: {TIMEOFPREVIOUSFIXSESSIONEND}
  Time of session change test: {TIMEOFSESSIONCHANGETEST}.]]>
 </Message>
</Group>
</Messages>
```

The table shows the correspondence between the events on which messages are sent and the parameters available for them:

Event	Description	Parameters
ConnectedToQuikServer	Successful connection to the QUIK server	Templates: {TIME}
CouldNotConnectToQuikServer	Error when connecting to the QUIK server	Templates: {TIME}, {DISCONNECTREASON}
DisconnectedFromQuikServer	Lost connection to the QUIK server	
DisconnectedFromQuikServerPendingTrans	Lost connection to the QUIK server. When disconnecting, there were transactions that had not been responded to	Templates: {TIME}, {DISCONNECTREASON}, {PENDINGTRANS} Tables: PendingTransList
FIXOutSessionDisconnected	Stop the flow of sending FIX messages to the client	Templates: {TIME}, {FIXSESSIONKEY}
FIXInSessionDisconnected	Stop the flow of incoming FIX messages	

Event	Description	Parameters
FIXSessionEstablished	Successful establishing of FIX session	
FIXSessionDropped	FIX session terminated by the Interface due to receiving an invalid incoming session counter	Templates: {TIME}, {FIXSESSIONKEY}, {DISCONNECTREASON}
FIXLogonFailed	Client request to establish a FIX session is rejected	
InvalidSecuritySpecification	Instrument from the client request is not found	Templates: {TIME}, {IDSOURCE}, {SECURITYID}, {SYMBOL}, {EXDESTINATION}
AmbiguouslySecuritySpecification	Instrument from the client request cannot be uniquely determined	
NewOrderRejected	Client request to place an order is rejected	Templates: {TIME}, {REJECTREASON}, {EMAILTAGS} Tables: EmailTagsList
ReplaceOrderRejected	Client request to amend an order is rejected	
CancelOrderRejected	Client request to cancel an order is rejected	
MarketDataRequestRejected	Client request to create the subscription for market information is rejected	
PossiblyIncorrectClearingTimeDetected	Incorrect session change time diagnosed	Templates: {TIMEOFPREVIOUSFIXSESSIONEND}, {TIMEOFSESSIONCHANGETEST}, {CONFIGUREDCLEARINGTIME}

2.12 Methods of instrument identification

Method of security identification in a message is set in tag [22] IDSource, which has the next valid values:

- 1 – by instrument CUSIP code;
- 2 – by instrument SEDOL code;
- 4 – by instrument ISIN-number;
- 5 – by instrument RIC code;
- 6 – by instrument ISO code;
- 8 – by instrument of the QUIK code;
- 100 – by instrument short name in the QUIK;
- 101 – by registration instrument number.

The security identifier is set in tag [48] SecurityID.

The short name of an instrument is set in tag [55] Symbol.

> **Features of filling the [48] SecurityID and [55] Symbol tag for OMS orders:**
>
> 1. **The value of the tag [48] SecurityID is transmitted as an instrument code if the instrument is not found and the tag [55] Symbol is not filled in.**
> 2. **The value of the tag [55] Symbol is used for searching for instrument by code in QUIK if ClientSecurityIDType=1 in [General] section and is transmitted as instrument identifier if the instrument is not found.**

If the list of instruments loaded by the Interface has two or more instruments with the same identifiers, the Interface will reject the message with the diagnostics: 'Ambiguously security specification'. In this case for the instrument identification it is necessary to point class code or market code in tag [100] ExDestination.

All the classes transmitted through the Interface may be classified according to the markets they are traded at. The list of classes may be set for every market in its settings section (see 2.6.16). The number of securities available for searching will be bounded with the classes and instruments of a certain market. If class code is identified in tag [100] the number of securities available for searching will be bounded with the class. Settings of parameters on markets and classes are given in 2.6.16-2.6.19).

For example:

```
22=8|100=MICEX|48=LKOH
```

2.13 Methods of trade account identification

2.13.1 Using trade account by default

Set the trade account for using by default in QUIK-FIX settings file (see 2.6.7). The value will be used if Account is not filled.

2.13.2 Using Account field

Type the client's trade account in the Account field. The value will be used in orders sending to the trade systems.

2.13.3 Using client code and trade account substitution

Set substitution 'ClientID = <Trade account>' in QUIK-FIX settings file (see 2.6.20). QUIK-FIX will put Account value instead of ClientID received from tags automatically.

2.13.4 Using trade accounts substitution

Set substitution '<Account_1> = <Account_2>' in QUIK-FIX settings file (see 2.6.20). QUIK-FIX will put Account_2 value instead of Account_1 received from tags.

2.14 Methods of client code identification

2.14.1 Using client identifier by default

Set the client identifier for using by default in QUIK-FIX settings file (see 2.6.7). The value will be used if ClientID is not filled.

2.14.2 Using ClientID field

Type the client's identifier in the ClientID field. The value will be used in orders sending to the trade systems.

2.14.3 Using the conversion of external client identifiers into QUIK client codes

Set substitution 'ClientID = <QUIK Client code>' in QUIK-FIX settings file (see 2.6.8). QUIK-FIX will search client code corresponding to the ClientID received from tags. The value will be used in orders sending to the trade systems.

2.15 SecTypes.xml file format

SecTypes.xml file is intended to set correspondences between instrument types and values transmitted in tag [167] SecurityType.

File format:

```
<SecTypes>
	<SecType
		FullName="<Full name of instrument type>"
		QuikType="<Code of instrument type in QUIK>"
		QuikSubType ="<Sub-type code of instrument in QUIK>"
		FIXType="<Value of tag [167] SecurityType>" />
	…
</SecTypes>
```

For example:

```
<SecTypes>
	<SecType
		FullName="Corporate bonds"
		QuikType="2"
		QuikSubType="4"
```

```
            FIXType="CORP" />
      <SecType
            FullName="Privileged share"
            QuikType="1"
            QuikSubType="2"
            FIXType="PS" />
</SecTypes>
```

3. Launch parameters

> 1. **The launch parameters for installing and removing the service are described in 2.3 and 2.5 respectively.**
> 2. **For the correct operation of the launch parameters, the user must have permissions to the directory with the installed service.**
> 3. **If it is necessary to start the Interface with the transfer of launch parameters as a service installed as a systemd service, use the q_start.h script, which is created in the directory when installing the service.**

3.1 -drop_history

Run the Inteface as a service to clear the storage file of multiday orders.

3.2 -new

Run the Interface as a service in the sent message storages deleting mode (clients' transaction history). The counters of incoming and outgoing messages are reset to 1.

3.3 -v

Run the Interface in console mode to display information about the Interface version.

Example of launch parameter:

```
[Executable file] -v
```

Interface version format displayed in console:

```
Version: <Major version>.<Minor version>.<Patch number>.<Build number>
```

3.4 -rollout

Run the Interface in console mode to generate initialization configuration files.

Example of launch parameter:

```
[Executable file] -rollout
```

As a result of executing the command, the following files will be created:

- qrypto.cfg;
- quikapi.ini;
- <executablename>.ini;
- sectypes.xml.

If the configuration files already exist in the directory, they will be saved and new ones will not be created.

4. Password encryption

The path to the setting values store file is specified in PSFile setting of [Files] section (see 2.6.1).

Passwords specified in the Interface settings can be encrypted. The password must not begin with the following characters: "$" and "#". If you use a password that begins with any of the above characters, change the first character.

To activate setting values encrypting functionality, add $ before the setting value in the **quikfix.ini** file.

For example:

```
Password=$secret
```

When attempting to use a password, the Interface encrypts and saves the password to the file specified in the **PSFile** setting of the [Files] section or to the **quikfix.ini** settings file in an encrypted form. The encrypted setting value is marked by # character at the start of a string.

> **When transferring the Interface to another computer or reinstalling the operational system, the encrypted setting values get unavailable for decrypting. In this case, the appropriate value must be reset.**